Perfect! Since your exam is tomorrow, we will prepare in exam-practical mode. No unnecessary theory and no waiting between steps.

I'll give you the exact commands in order. You just copy, paste and execute them in Git Bash. I'll also explain briefly what each command does so you can answer viva questions.

Experiment 1 – Complete practical steps

Step 1. Configure Git (first-time setup only)

Open Git Bash and execute:

git config --global user.name "Your Name"

git config --global user.email "your-email@gmail.com"

Use your actual GitHub name and registered email.

Check configuration:

git config --global --list

Step 2. Create a project folder

mkdir DataScienceProject

cd DataScienceProject

- mkdir = create a folder.
- cd = enter a folder.

Initialize Git:

git init

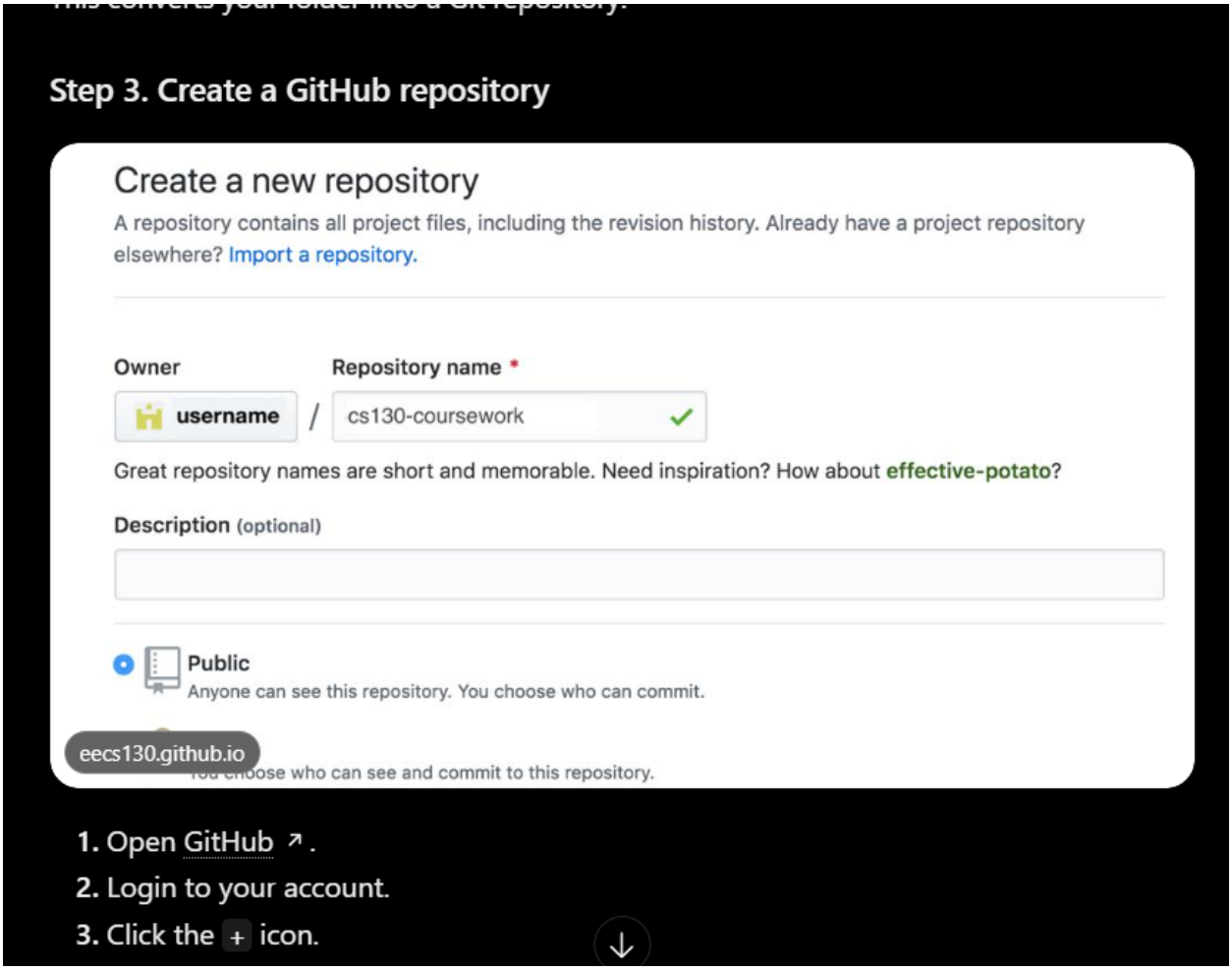
This converts your folder into a Git repository.

Step 3. Create a GitHub repository

Getting Started with GitHub

eecs130.github.io

1. Open GitHub.
2. Login to your account.
3. Click the + icon.
4. Select New repository.

5. 

   Repository name: DataScienceProject.
6. Select Public.
7. Do not select README, .gitignore or License.
8. Click Create repository.

Step 4. Connect local repository to GitHub

Copy your repository HTTPS URL from GitHub.

It will look like:

https://github.com/USERNAME/DataScienceProject.git

Execute (replace with your actual URL):

git remote add origin https://github.com/USERNAME/DataScienceProject.git

Set the main branch name as master:

git branch -M master

Push the initial repository:

git push -u origin master

If GitHub asks for authentication, complete the browser sign-in.

Step 5. Create program1.py

Execute:

notepad program1.py

If Windows asks to create a new file, click Yes.

Write this Python program:

num = int(input("Enter a number: "))for i in range(1, 11): print(num, "x", i, "=", num * i)

Save the file and close Notepad.

Example output:

Enter a number: 5

5 x 1 = 5

5 x 2 = 10

5 x 3 = 15

5 x 4 = 20

5 x 5 = 25

...

5 x 10 = 50

Step 6. Add, commit and push program1.py

Execute these commands one by one:

git status

Shows modified and untracked files.

git add program1.py

Adds the file to the staging area.

git commit -m "Add multiplication table program"

Saves the changes as a commit.

git push origin master

Uploads the changes to GitHub.

Now refresh your GitHub repository. You should see program1.py.

Step 7. Create a new branch named GIT-EXAM

git checkout -b GIT-EXAM

This creates a new branch and switches to it.

Check your current branch:

git branch

Expected output:

* GIT-EXAM

  master

The * indicates your current branch.

Step 8. Create program2.py

notepad program2.py

Write this program:

a = int(input("Enter first number: "))b = int(input("Enter second number: "))if a > b: print("Greatest number is:", a)elif b > a: print("Greatest number is:", b)else: print("Both numbers are equal")

Save and close Notepad.

Example output:

Enter first number: 20

Enter second number: 10

Greatest number is: 20

Step 9. Commit and push the new branch

Execute:

git add program2.py

git commit -m "Add greatest of two numbers program"

git push -u origin GIT-EXAM

Now your new branch is also uploaded to GitHub.

Step 10. Demonstrate differences between branches

First, switch to master:

git checkout master

List files:

ls

Expected:

program1.py

Now switch to GIT-EXAM:

git checkout GIT-EXAM

List files:

ls

Expected:

program1.py

program2.py

Compare the branches:

git diff --name-status master GIT-EXAM

Expected output:

A program2.py

A means Added.

Experiment 1 completed

Important commands for your exam

Command	Purpose
git init	Initialize repository
git status	Check file status
git add	Add files
git commit	Save changes
git push	Upload changes
git pull	Download remote changes
git branch	List branches
git checkout -b	Create and switch branch
git checkout	Switch branch
git diff	Compare changes
git remote -v	Show remote repository

Viva questions

1. What is Git? A distributed version control system.
2. What is GitHub? An online platform for hosting Git repositories.
3. What is a repository? A folder that stores project files and their version history.
4. What is a commit? A saved snapshot of changes.
5. What is a branch? An independent line of development.
6. What is push? Uploading local commits to a remote repository.
7. What is git init? It initializes a Git repository.
8. What is git add? It stages files for committing.

Exam tip: Remember the command sequence:

init → remote add → add → commit → push → branch → add → commit → push → diff

Next, send me Experiment 2. I'll prepare it in the same direct, copy-paste practical format, with expected outputs and viva questions.

Experiment 2: Git and GitHub using Git Bash

Windows + Git Bash

Exam practical

This experiment is almost the same as Experiment 1. Only the Python programs and the final Pull Request step are different.

Important: You can use a new repository named DataScienceProject2 to keep this experiment separate.

Step 1. Configure Git

If you already configured Git in Experiment 1, skip this step.

git config --global user.name "Your Name"

git config --global user.email "your-email@gmail.com"

Step 2. Create project and initialize repository

mkdir DataScienceProject2

cd DataScienceProject2

git init

Set branch name:

git branch -M master

Step 3. Create GitHub repository

1. Open GitHub.
2. Login.
3. Click + → New repository.
4. Repository name: DataScienceProject2.
5. Select Public.
6. Do not initialize with README.
7. Click Create repository.

Connect GitHub:

git remote add origin https://github.com/USERNAME/DataScienceProject2.git

Replace USERNAME with your GitHub username.

Push initial branch:

git push -u origin master

Step 4. Create program1.py (Even or Odd)

Execute:

notepad program1.py

Click Yes if asked to create a file.

Write this code:

num = int(input("Enter a number: "))if num % 2 == 0: print("Number is Even")else: print("Number is Odd")

Save and close Notepad.

Output:

Enter a number: 10

Number is Even

Another example:

Enter a number: 7

Number is Odd

Step 5. Commit and push program1.py

Execute one by one:

git status

git add program1.py

git commit -m "Add even odd program"

git push origin master

Now open your GitHub repository and check that program1.py is visible.

Step 6. Create a new branch GIT-EXAM

git checkout -b GIT-EXAM

Check branches:

git branch

Expected:

* GIT-EXAM

  master

Step 7. Create program2.py (Reverse of a string)

Execute:

notepad program2.py

Write this Python code:

text = input("Enter a string: ")reverse = text[::-1]print("Reverse string:", reverse)

Save and close Notepad.

Output:

Enter a string: HELLO

Reverse string: OLLEH

Explanation:

- text stores the original string.
- [::-1] reverses the string.
- print() displays the reversed string.

Step 8. Commit and push changes

git add program2.py

git commit -m "Add reverse string program"

git push -u origin GIT-EXAM

Now the GIT-EXAM branch is available on GitHub.

Step 9. Demonstrate difference between branches

Switch to master:

git checkout master

Check files:

ls

Expected:

program1.py

Switch to GIT-EXAM:

git checkout GIT-EXAM

Check files:

ls

Expected:

program1.py

program2.py

Compare branches:

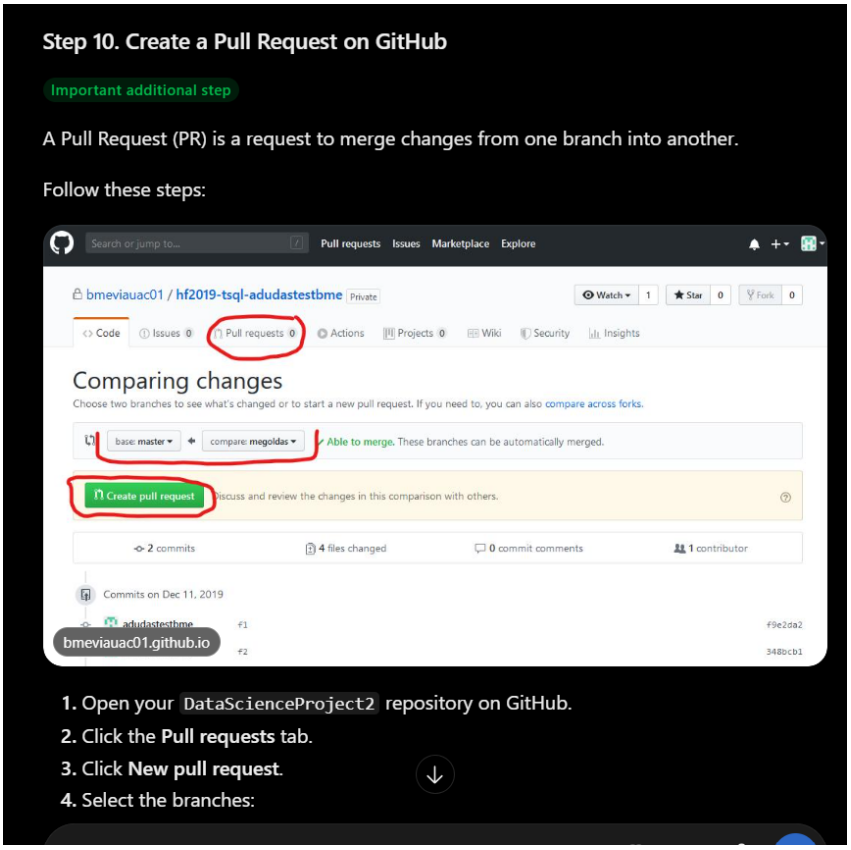
git diff --name-status master GIT-EXAM

Expected:

A program2.py

This shows that program2.py was added in GIT-EXAM.

Step 10. Create a Pull Request on GitHub



Important additional step

A Pull Request (PR) is a request to merge changes from one branch into another.

Follow these steps:

[Submitting your work (GitHub) - BMEVIAUAC16 Data-Driven Software Development Lab](bmeviauac01.github.io)

1. Open your DataScienceProject2 repository on GitHub.
2. Click the Pull requests tab.
3. Click New pull request.
4. Select the branches:

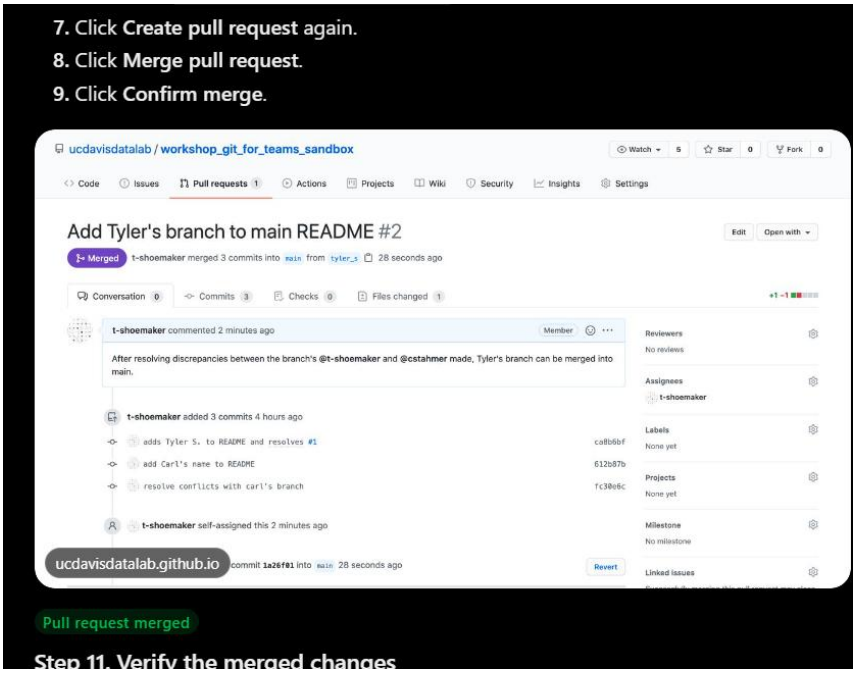
Base branch (destination)

master

Compare branch (source)

GIT-EXAM

Merge GIT-EXAM into master

5. Click Create pull request.
6. Enter title: Merge GIT-EXAM into master.
7. Click Create pull request again.
8. Click Merge pull request.
9. Click Confirm merge.
10. 

[6. Open and Merge a Pull Request — Reproducible Research for Teams with GitHub ucdavisdatalab.github.io]

Pull request merged

Step 11. Verify the merged changes

Return to Git Bash:

git checkout master

Download merged changes:

git pull origin master

Check files:

ls

Expected output:

program1.py

program2.py

Now both programs are available in master.

Important viva questions

Question	Answer
What is Git?	Version control system.
What is GitHub?	Online Git repository hosting platform.
What is a branch?	Separate development line.
What is a Pull Request?	Request to merge changes from one branch into another.
What is merging?	Combining changes from different branches.
What is git diff?	Command used to compare changes.
What is git checkout -b?	Creates and switches to a new branch.
What is [::-1]?	Reverses a string in Python.
What is %?	Modulus operator, returns remainder.

Experiment 1 vs Experiment 2

Particular	Experiment 1	Experiment 2
Program 1	Multiplication table	Even/Odd
Program 2	Greatest of two numbers	Reverse string
Branch	GIT-EXAM	GIT-EXAM
Pull Request	Not required	Required

Exam revision: Complete command sequence

```
git config --global user.name "Your Name"
git config --global user.email "your-email@gmail.com"

mkdir DataScienceProject2
cd DataScienceProject2

git init
git branch -M master

git remote add origin https://github.com/USERNAME/DataScienceProject2.git
git push -u origin master

notepad program1.py

git add program1.py
git commit -m "Add even odd program"
git push origin master

git checkout -b GIT-EXAM

notepad program2.py

git add program2.py
git commit -m "Add reverse string program"
git push -u origin GIT-EXAM
```

git checkout master

ls

git checkout GIT-EXAM

ls

git diff --name-status master GIT-EXAM

Then create and merge the Pull Request on GitHub.

One important exam note: If your college repository uses main instead of master, replace every master in the commands and Pull Request settings with main.

Send Experiment 3. We will continue in the same format so you can revise all experiments before tomorrow's exam.

Experiment 3: Git Branching, Pulling, Conflicts and Pull Request

Windows + Git Bash

Exam practical

This experiment is a little more advanced than Experiments 1 and 2 because it introduces two branches and merge conflicts.

Don't worry. Follow the commands in the exact order given below.

1. Understand what we are doing

We will create three branches:

- master – original branch.
- GIT-EXAM-1 – first branch containing palindrome program.
- GIT-EXAM-2 – second branch where we pull changes from the first branch and modify the multiplication/square program.

Our branch structure:

master

Original branch

GIT-EXAM-1

Palindrome + squares program

GIT-EXAM-2

Pulled changes + modified program

Step 2. Configure Git

Skip this if already configured in Experiment 1.

git config --global user.name "Your Name"

git config --global user.email "your-email@gmail.com"

Step 3. Create project and initialize repository

mkdir DataScienceProject3

cd DataScienceProject3

git init

git branch -M master

Create a new repository on GitHub:

- Repository name: DataScienceProject3
- Visibility: Public
- Do not initialize with README.

Connect your GitHub repository:

git remote add origin https://github.com/USERNAME/DataScienceProject3.git

git push -u origin master

Replace USERNAME with your GitHub username.

Step 4. Create branch GIT-EXAM-1

git checkout -b GIT-EXAM-1

Check:

git branch

Expected:

* GIT-EXAM-1

  master

Step 5. Create program2.py (Palindrome string)

notepad program2.py

Write this code:

text = input("Enter a string: ")if text == text[::-1]:   print("String is Palindrome")else:
print("String is not Palindrome")

Example:

Enter a string: madam

String is Palindrome

A palindrome reads the same forward and backward.

Step 6. Create program1.py (Squares from 1 to 10)

notepad program1.py

Write:

for i in range(1, 11):   print("Square of", i, "=", i * i)

Expected output:

Square of 1 = 1

Square of 2 = 4

Square of 3 = 9

Square of 4 = 16

Square of 5 = 25

Square of 6 = 36

Square of 7 = 49

Square of 8 = 64

Square of 9 = 81

Square of 10 = 100

Save both files.

Step 7. Add, commit and push GIT-EXAM-1

The assignment says program2 in step (c), without an extension. To follow it literally, rename the file if you created it as program2.py:

mv program2.py program2

Now commit both files:

git add .

git commit -m "Add palindrome and square programs"

git push -u origin GIT-EXAM-1

Check your GitHub repository. Select branch GIT-EXAM-1. You should see:

program1.py

program2

Step 8. Create GIT-EXAM-2 branch

First switch to master:

git checkout master

Create the second branch:

git checkout -b GIT-EXAM-2

Push it to GitHub:

git push -u origin GIT-EXAM-2

At this stage, GIT-EXAM-2 starts from master and does not yet contain the files from GIT-EXAM-1.

Step 9. Pull changes from GIT-EXAM-1

Execute:

git pull origin GIT-EXAM-1

This brings the first branch's committed changes into your current branch, GIT-EXAM-2.

Check:

ls

Expected:

program1.py

program2

Now GIT-EXAM-2 also contains the first branch's files.

Step 10. Modify program1.py in GIT-EXAM-1

This is important for demonstrating a conflict.

Switch back:

git checkout GIT-EXAM-1

Open the file:

notepad program1.py

Replace the previous code with:

for i in range(1, 11):   print(i, "square is", i ** 2)

Save the file.

Commit and push:

git add program1.py

git commit -m "Modify square program in branch 1"

git push origin GIT-EXAM-1

Now branch 1 has a new version of program1.py.

Step 11. Modify the same program in GIT-EXAM-2

Switch to the second branch:

git checkout GIT-EXAM-2

Open:

notepad program1.py

Replace the code with a different version:

for i in range(1, 11):   print("Number:", i, "Square:", i * i)

Save the file.

Commit and push:

git add program1.py

git commit -m "Modify square program in branch 2"

git push origin GIT-EXAM-2

Why did we modify the same file in both branches?

Because Git needs different changes to the same part of a file to demonstrate a merge conflict.

Step 12. Pull the latest GIT-EXAM-1 changes into GIT-EXAM-2

Make sure you are on GIT-EXAM-2:

git branch

Then execute:

git pull origin GIT-EXAM-1

Merge conflict expected

You may see:

CONFLICT (content): Merge conflict in program1.py

Automatic merge failed; fix conflicts and then commit the result.

This is not a software error. We intentionally created this situation for the experiment.

Step 13. Resolve the merge conflict

Open:

notepad program1.py

You may see something similar to:

<<<<<<< HEAD

```
for i in range(1, 11):
    print("Number:", i, "Square:", i * i)
=======
for i in range(1, 11):
    print(i, "square is", i ** 2)
>>>>>>> ...
```

These markers show the two different versions.

Remove all conflict markers and keep one final version:

```
for i in range(1, 11):    print("Number:", i, "Square:", i * i)
```

Save and close the file.

Now execute:

```
git add program1.py
git commit -m "Resolve merge conflict"
git push origin GIT-EXAM-2
```

Conflict resolved!

Remember these three markers for your viva:

- <<<<<<< HEAD – your current branch changes.
- ======= – separates the two versions.
- >>>>>>> – incoming branch changes.

Step 14. Create program2.py in GIT-EXAM-2

The assignment separately asks for a file named program2.py in the second branch.

notepad program2.py

Write:

text = input("Enter a string: ")if text == text[::-1]: print("Palindrome")else: print("Not Palindrome")

Save and close.

Commit and push:

git add program2.py

git commit -m "Add palindrome program in branch 2"

git push origin GIT-EXAM-2

Step 15. Demonstrate differences between all three branches

First check master:

git checkout master

ls

Expected:

Master is empty because we created the project without adding files to it.

Now check GIT-EXAM-1:

git checkout GIT-EXAM-1

ls

Expected:

program1.py

program2

Now check GIT-EXAM-2:

git checkout GIT-EXAM-2

ls

Expected:

program1.py

program2

program2.py

Compare branches:

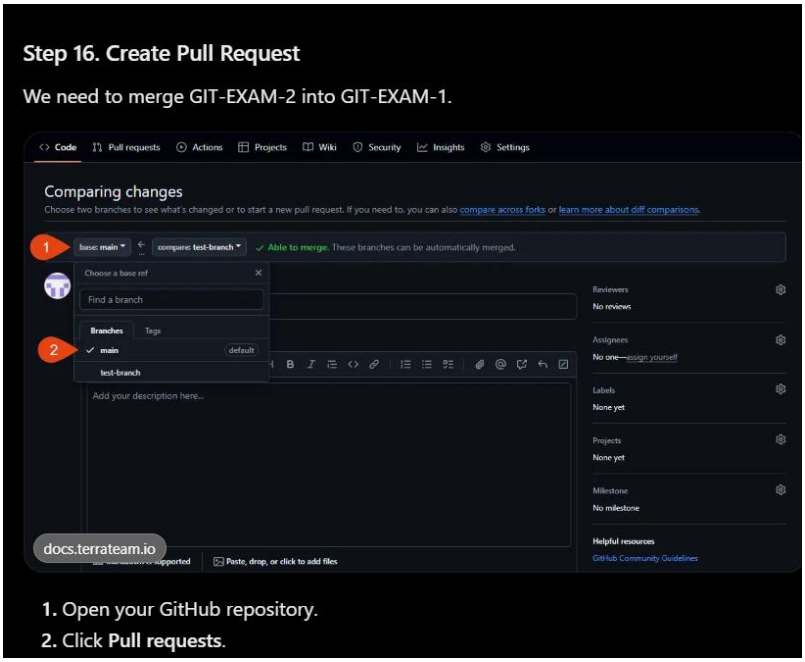
git diff --name-status master GIT-EXAM-1

git diff --name-status master GIT-EXAM-2

git diff --name-status GIT-EXAM-1 GIT-EXAM-2

These commands show the differences between branches.

Step 16. Create Pull Request



We need to merge GIT-EXAM-2 into GIT-EXAM-1.

[Pull Requests and Triggers | Terrateam]

[docs.terrateam.io]

1. Open your GitHub repository.
2. Click Pull requests.
3. Click New pull request.
4. Select these branches:

Base (destination)

GIT-EXAM-1

Compare (source)

GIT-EXAM-2

GIT-EXAM-2 → GIT-EXAM-1

5. Click Create pull request.
6. Title: Merge GIT-EXAM-2 into GIT-EXAM-1.
7. Click Create pull request.
8. Click Merge pull request.
9. Click Confirm merge.

Pull Request merged

Important viva questions

Question	**Answer**
What is Git?	Version control system.
What is branching?	Creating a separate line of development.
What is merging?	Combining changes from two branches.
What is git pull?	Fetches and integrates remote changes.
What is merge conflict?	When Git cannot automatically combine conflicting changes.
How do you resolve conflicts?	Manually edit the file, remove conflict markers, add and commit.
What is HEAD?	Reference to the currently checked-out commit.

What is a Pull Request?	Request to merge changes into another branch.
What is git diff?	Compares changes between files or branches.
What is a palindrome?	A string that reads the same forward and backward.

Quick revision: Commands to remember

git init

git branch -M master

git remote add origin URL

git push -u origin master

git checkout -b GIT-EXAM-1

git add .

git commit -m "First commit"

git push -u origin GIT-EXAM-1

git checkout master

git checkout -b GIT-EXAM-2

git pull origin GIT-EXAM-1

git checkout GIT-EXAM-1

git add .

git commit -m "Modify program"

git push origin GIT-EXAM-1

git checkout GIT-EXAM-2

git add .

git commit -m "Modify program"

git push origin GIT-EXAM-2

git pull origin GIT-EXAM-1

Resolve conflict manually

git add .

git commit -m "Resolve conflict"

git push origin GIT-EXAM-2

Then create the Pull Request on GitHub.

Very important exam notes

1. Always check your current branch using git branch.
2. Before modifying a file, check which branch you are working on.
3. A merge conflict is intentional in this experiment.
4. Never leave conflict markers in the final Python file.
5. master and main are alternative default branch names. Use the one specified in your assignment/repository.

One correction to remember: In step (c), your assignment writes program2 without .py, while step (h) specifically says program2.py. I have followed those names separately.

Send Experiment 4. We will finish all five Git Bash experiments first, then move to Ubuntu.

Experiment 4: Git and GitHub using Git Bash (.gitignore)

Windows + Git Bash

Exam practical

This experiment is similar to Experiment 3, but here we learn an important Git concept called `.gitignore`.

1. What is .gitignore?

`.gitignore` is a special file that tells Git which files or folders should be ignored.

For example, suppose your project contains:

```
DataScienceProject4/
|
 ├── program1.py
 ├── program2
 ├── .gitignore
|
```

└── IGNORE/

    ├── test1.txt

    └── test2.txt

```

We want GitHub to contain:

- `program1.py`
- `program2`
- `.gitignore`

We do NOT want GitHub to contain:

- `IGNORE/`
- `test1.txt`
- `test2.txt`

## Step 2. Configure Git

If already configured, skip this step.

```
git config --global user.name "Your Name"
git config --global user.email "your-email@gmail.com"
```
```

Step 3. Create a project and initialize Git

```
mkdir DataScienceProject4
cd DataScienceProject4
git init
git branch -M master
```

Create a new repository on [GitHub](https://github.com?utm_source=chatgpt.com).

- Repository name: `DataScienceProject4`
- Select Public.
- Do not initialize with README.

Connect GitHub:

```
git remote add origin https://github.com/USERNAME/DataScienceProject4.git
```

Push initial branch:

```
git push -u origin master
```

Replace `USERNAME` with your GitHub username.

Step 4. Create branch GIT-EXAM-1

```
git checkout -b GIT-EXAM-1
```

Check:

```
git branch
```

Expected:

```
* GIT-EXAM-1
  master
```

Step 5. Create program2 (Palindrome program)

```
notepad program2
```

```

Click Yes if asked to create a file.

Write:

```
text = input("Enter a string: ")if text == text[::-1]:   print("String is Palindrome")else:
print("String is not Palindrome")
```

Save and close Notepad.

Example output:

```
Enter a string: madam
String is Palindrome
```

Step 6. Create program1.py (Squares of numbers)

```
notepad program1.py
```
```

Write:

```
for i in range(1, 11):    print("Square of", i, "=", i * i)
```

Save and close.

Expected output:

```
Square of 1 = 1
Square of 2 = 4
Square of 3 = 9
Square of 4 = 16
Square of 5 = 25
Square of 6 = 36
Square of 7 = 49
Square of 8 = 64
Square of 9 = 81
Square of 10 = 100
```

Step 7. Add, commit and push the programs

```
```

```
git add program1.py program2
git commit -m "Add palindrome and square programs"
git push -u origin GIT-EXAM-1
```

Open GitHub and select the `GIT-EXAM-1` branch.

You should see:

```
program1.py
program2
```

Step 8. Create IGNORE directory

Now we will create a folder named `IGNORE`.

Execute:

```
mkdir IGNORE
```

Enter the folder:

```
cd IGNORE
```

Create first text file:

```
notepad test1.txt
```

Write anything, for example:

```
This is test file 1
```

Save and close.

Create second file:

```
notepad test2.txt
```

Write:

```
This is test file 2
```

Save and close.

Return to the main project folder:

```
cd ..
```

Check folder structure:

```
ls
```

Expected:

```
IGNORE
program1.py
program2
```

Step 9. Create .gitignore file

Execute:

```
notepad .gitignore
```

Write exactly:

```
IGNORE/
```

Save and close.

[Git & GitHub Survival Guide for Data Engineers: The Ultimate Command Bible | by Abhishek Kumar Gupta | Medium | Tech with Abhishek](https://images.openai.com/static-rsc-4/WGCFvzDln8i-8jPQ_6y62RG_vXAnkPUV0IX-hBwvPkos5nHv_mTHlD6xafWTIHWR6OBsEdGcNU_2K7rzK0fFyFguwQvz1Q0M-GoQsYx7uhaN31x-pUaBT8WNjrUOPBkC-iPCJRE02TDZINqTrfT0PMIHpeP5plNpFOewRaUi8aA?purpose=inline)

[medium.com](https://medium.com/tech-with-abhishek/%EF%B8%8F-git-github-survival-guide-for-data-engineers-the-ultimate-command-bible-8441308ce209?utm_source=chatgpt.com)

Meaning: Git will ignore all files and folders inside the `IGNORE` directory.

Step 10. Check ignored files

Execute:

```
git status
```

You should see `.gitignore` as an untracked file.

The `IGNORE` directory should not appear as an untracked folder.

To verify ignored files, execute:

```
git status --ignored
```

Expected output will contain something similar to:

```
!! IGNORE/
```

`!!` means Git is ignoring that directory.

Step 11. Commit and push the changes

Add `.gitignore`:

```
git add .gitignore
```

Commit:

```
git commit -m "Add gitignore to ignore IGNORE directory"
```

Push:

```
git push origin GIT-EXAM-1
```

Step 12. Verify on GitHub

Open your GitHub repository.

1. Select the `GIT-EXAM-1` branch.
2. Refresh the page.

3. Check the files.

Expected GitHub contents:

`program1.py`

Pushed

`program2`

Pushed

`.gitignore`

Pushed

`IGNORE/`

Ignored

The IGNORE folder and its two text files will not be uploaded.

Important viva questions

Question	Answer

What is Git?	A version control system.
What is GitHub?	An online platform for Git repositories.
What is `.gitignore`?	A file used to ignore specified files and folders.
Why do we use `.gitignore`?	To prevent unwanted files from being tracked.
What is `IGNORE/`?	A rule to ignore the IGNORE directory and its contents.
What is `git status`?	Shows the current repository status.
What is `git status --ignored`?	Displays ignored files and folders too.
What does `!!` mean?	The file or directory is ignored.
What is `git add`?	Adds files to the staging area.
What is `git push`?	Uploads commits to GitHub.

Quick revision: All commands

```
mkdir DataScienceProject4
cd DataScienceProject4

git init
git branch -M master

git remote add origin https://github.com/USERNAME/DataScienceProject4.git
git push -u origin master

git checkout -b GIT-EXAM-1

notepad program2
```

notepad program1.py

git add program1.py program2

git commit -m "Add programs"

git push -u origin GIT-EXAM-1

mkdir IGNORE

cd IGNORE

notepad test1.txt

notepad test2.txt

cd ..

notepad .gitignore

```

Write `IGNORE/` inside `.gitignore`.

Continue:

```

git status

git status --ignored

git add .gitignore

git commit -m "Add gitignore"

git push origin GIT-EXAM-1

```

Experiment 4 completed

Important viva point: `.gitignore` does not remove files that Git is already tracking. In this experiment, we create the IGNORE directory after initializing Git and do not add it before applying the ignore rule.

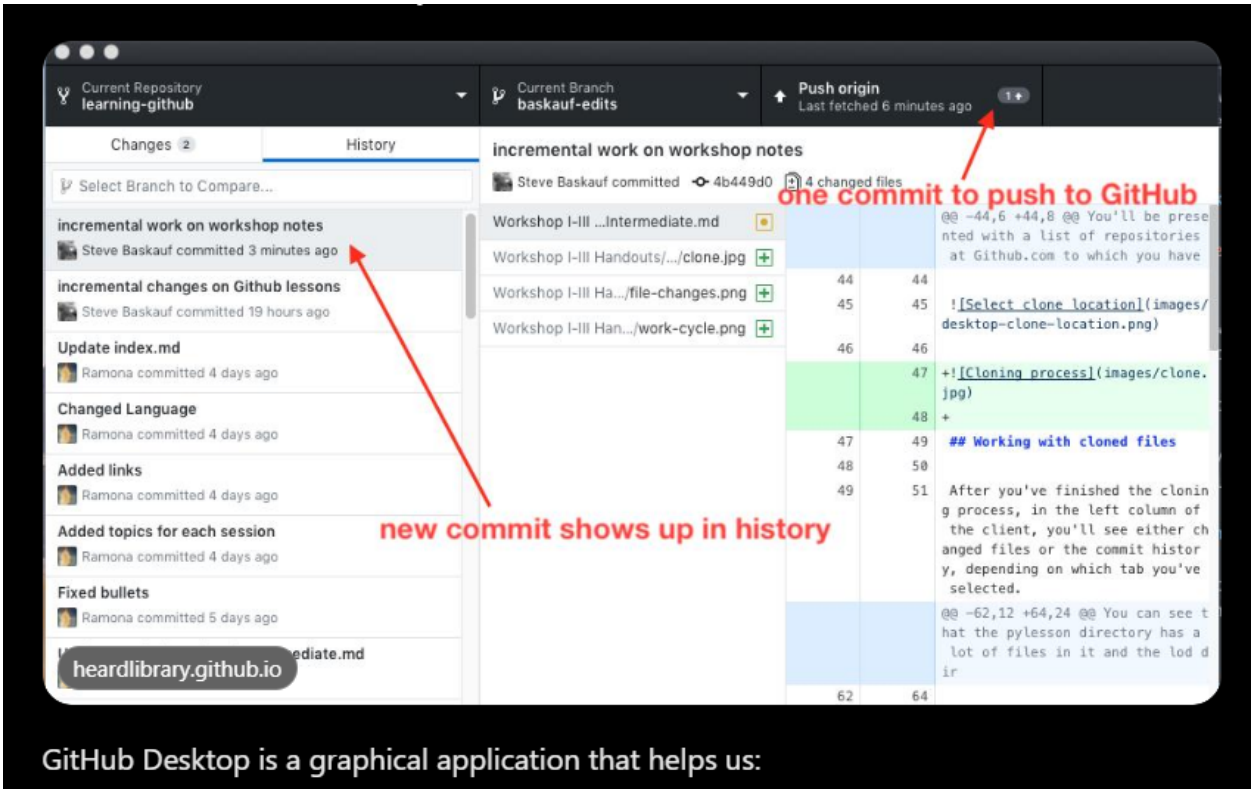
Send me Experiment 5. After that, we will start Ubuntu experiments from the absolute basics, including opening the terminal and learning Linux commands.
```

Experiment 5: GitHub Desktop – Git Repository, Branching and .gitignore

Windows – GitHub Desktop

Exam practical

Important difference: Experiments 1–4 use Git Bash commands. Experiment 5 uses GitHub Desktop, so we will perform most operations by clicking buttons instead of typing Git commands.

1. **What is GitHub Desktop?**
2. 

[Cloning and the GitHub desktop client | Digital Education Resources - Vanderbilt Libraries Digital Lab]

[heardlibrary.github.io]

GitHub Desktop is a graphical application that helps us:

- Create Git repositories.
- Upload projects to GitHub.
- Create branches.

- Commit changes.
- Push changes.
- Manage .gitignore.

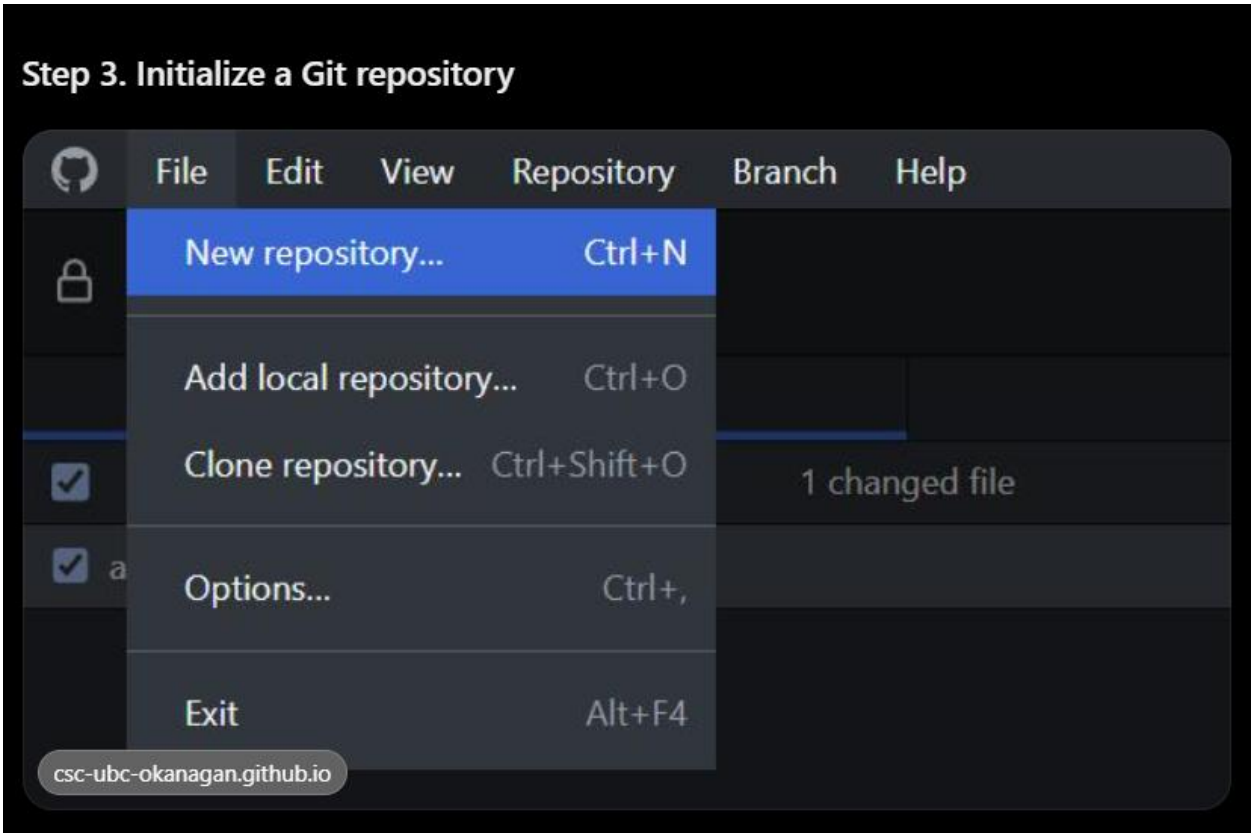
Step 2. Open GitHub Desktop

1. Click the Windows Start button.
2. Search for GitHub Desktop.
3. Open the application.
4. Sign in using your GitHub account.

If it is not installed, download it here:

[Download GitHub Desktop](#).

Step 3. Initialize a Git repository



[Getting started with GitHub]

[csc-ubc-okanagan.github.io]

1. Open GitHub Desktop.
2. Click File.
3. Select New repository.

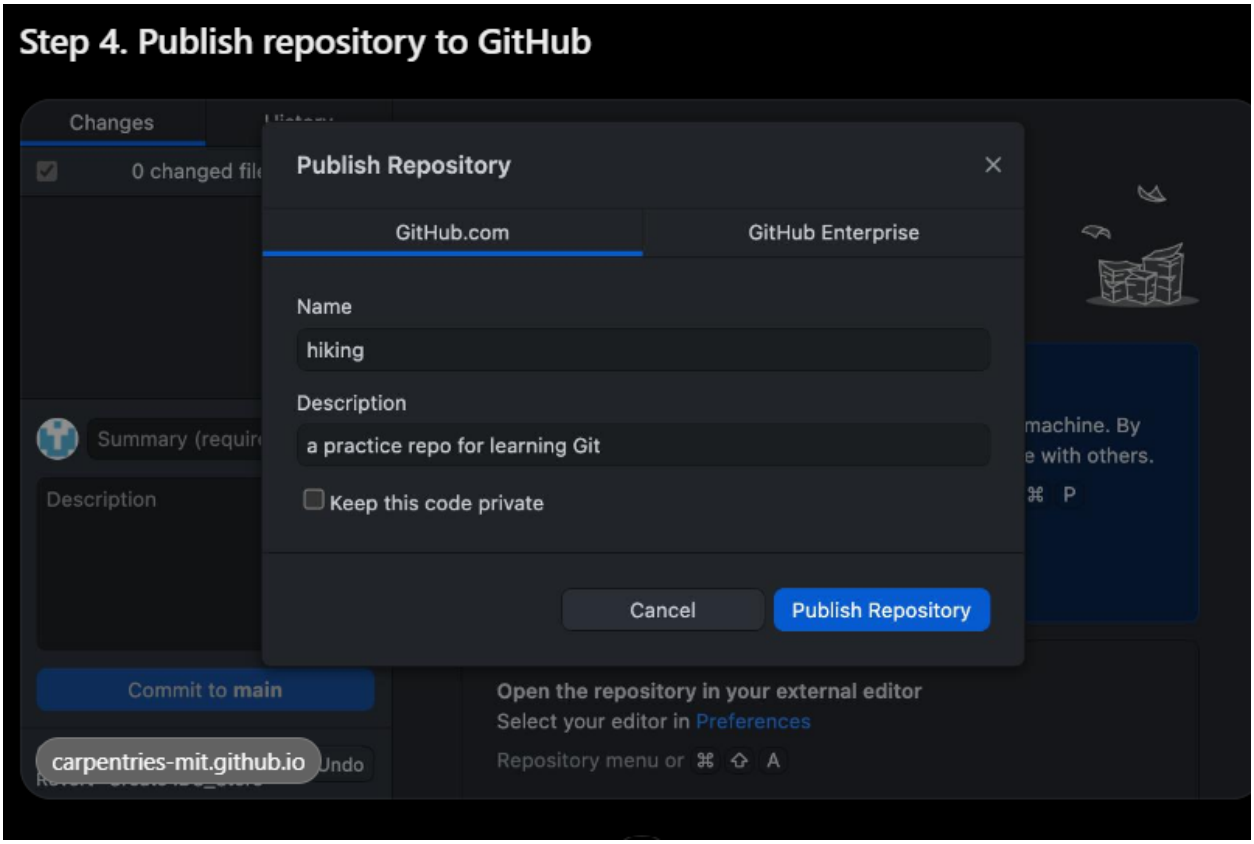
Fill in the details:

Field	**Value**
Name	DataScienceProject5
Description	Git practical experiment
Local path	Choose Desktop
Initialize with README	No
Git ignore	None
License	None

4. Click Create repository.

Your local Git repository is now created.

Step 4. Publish repository to GitHub



[Version Control with Git and GitHub — Version Control with Git and GitHub

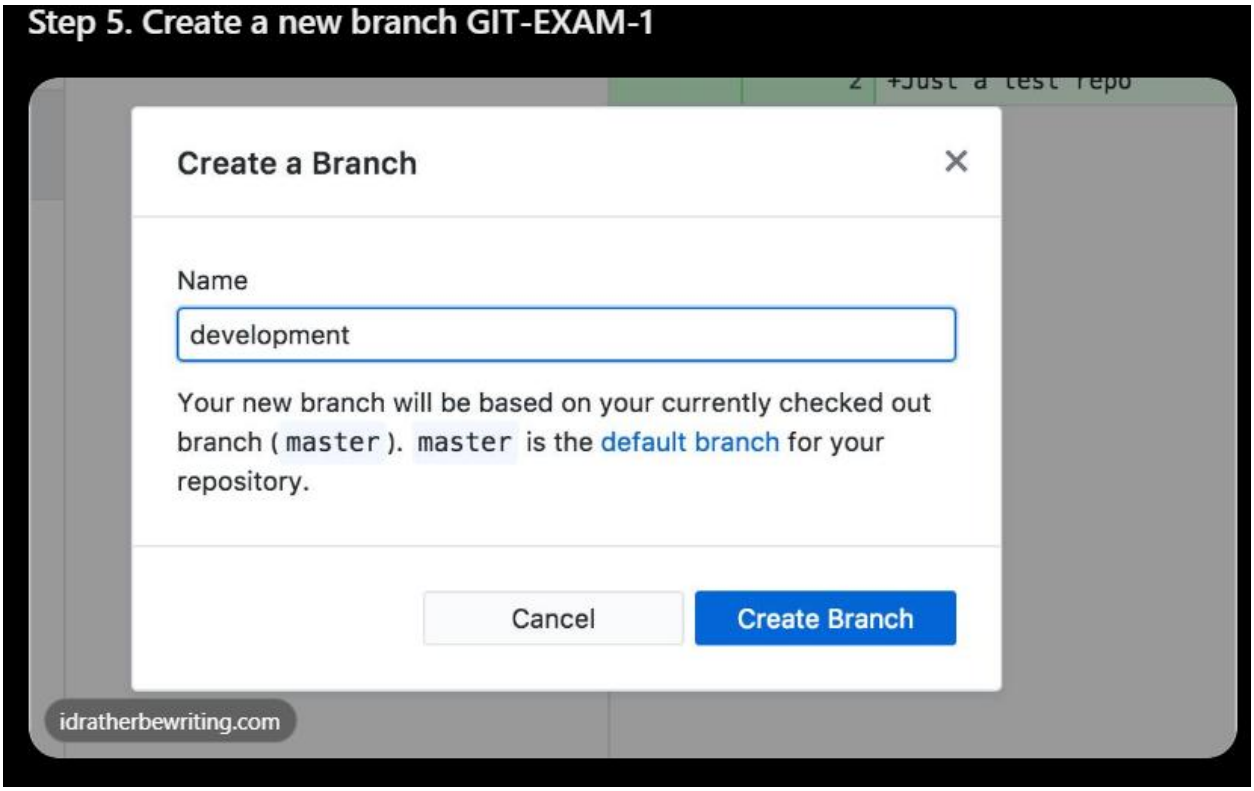
carpentries-mit.github.io]

1. Find the Publish repository button.
2. Click it.
3. Repository name: DataScienceProject5.
4. Choose Public.
5. Click Publish repository.

Your repository is now uploaded to GitHub.

To verify, click View on GitHub.

Step 5. Create a new branch GIT-EXAM-1



[Activity: Use the GitHub Desktop client | I'd Rather Be Writing Blog and API doc course](idratherbewriting.com)

1. Find the Current Branch option.
2. Click it.
3. Select New Branch.
4. Enter branch name:

GIT-EXAM-1

5. Click Create Branch.

Now you are working in the GIT-EXAM-1 branch.

Step 6. Create program2 (Palindrome program)

Open your project folder.

In GitHub Desktop:

1. Click Repository.
2. Select Show in Explorer.

3. Right-click inside the folder.
4. Select New → Text Document.
5. Name it program2 (without .txt).

If Windows hides extensions, enable View → Show → File name extensions in File Explorer.

Open the file and write:

text = input("Enter a string: ")if text == text[::-1]: print("String is Palindrome")else: print("String is not Palindrome")

Save the file.

Example output:

Enter a string: madam

String is Palindrome

Step 7. Create program1.py (Squares from 1 to 10)

Inside the same project folder:

1. Right-click → New → Text Document.
2. Rename it program1.py.
3. Open it using Notepad.

Write:

for i in range(1, 11): print("Square of", i, "=", i * i)

Save the file.

Expected output:

Square of 1 = 1

Square of 2 = 4

Square of 3 = 9

Square of 4 = 16

Square of 5 = 25

Square of 6 = 36

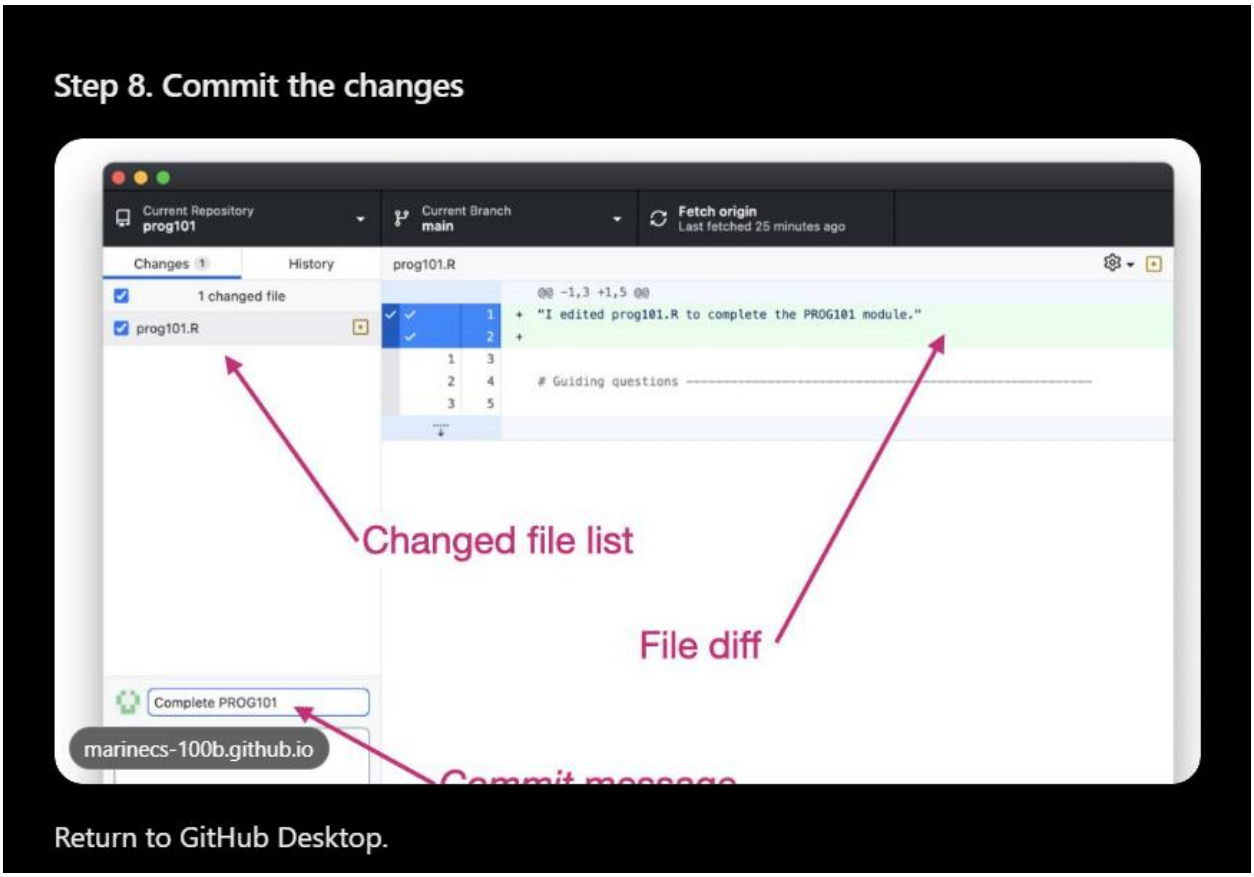
Square of 7 = 49

Square of 8 = 64

Square of 9 = 81

Square of 10 = 100

Step 8. Commit the changes



[Intro to Marine Data Science – Module Setup and Submission]

[marinecs-100b.github.io]

Return to GitHub Desktop.

You should see both files in the Changes tab.

1. Select both files.
2. In the Summary box, enter:

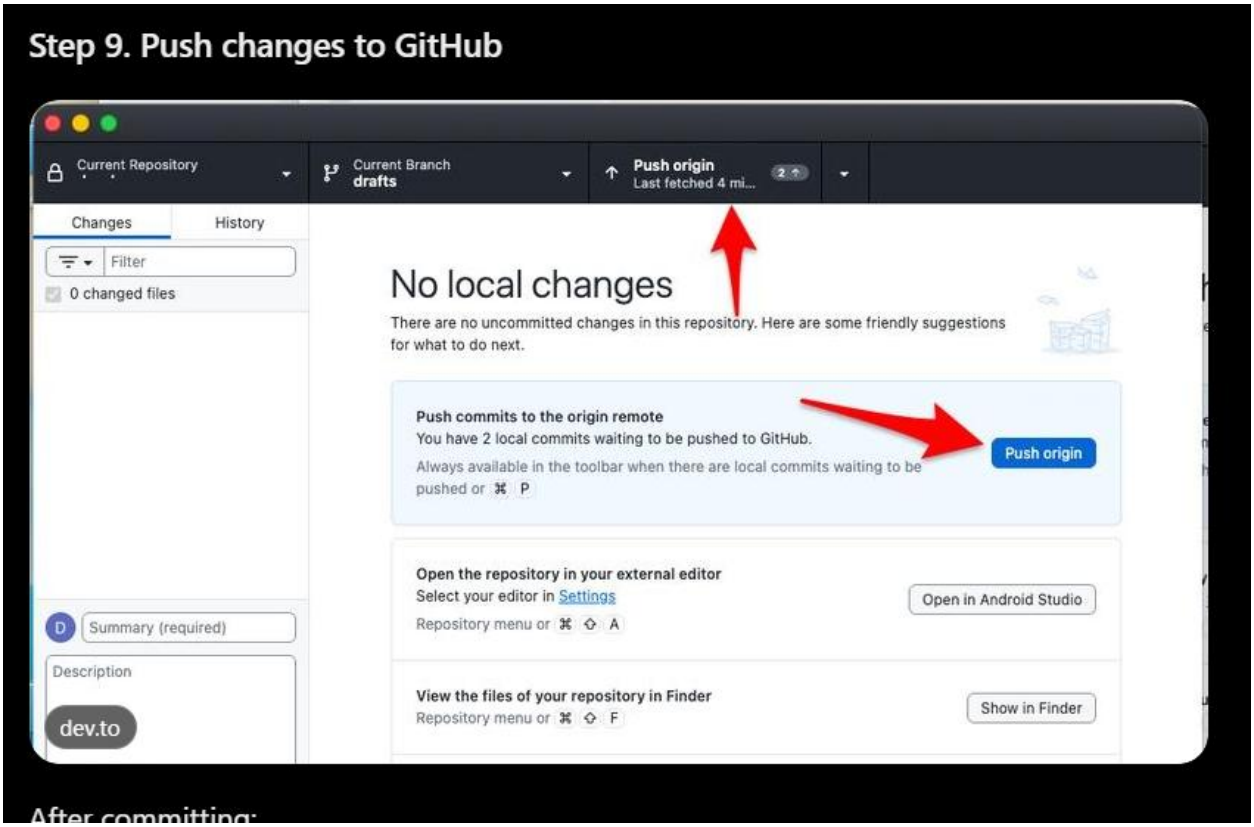
Add palindrome and square programs

3. Click Commit to GIT-EXAM-1.

What is commit?

Commit means saving a snapshot of your project.

Step 9. Push changes to GitHub



[GitHub Desktop | The Undo Button for Vibe Coders. - DEV Community

dev.to](dev.to)

After committing:

1. Click Push origin at the top.
2. Wait for the upload to complete.
3. Open GitHub.
4. Select branch GIT-EXAM-1.

You should see:

program1.py

program2

Step 10. Create IGNORE directory

Now create a folder that we do not want to upload.

1. Open your project folder using File Explorer.
2. Right-click → New → Folder.
3. Name it:

IGNORE

Open the IGNORE folder.

Create two text files:

File 1: test1.txt

This is test file 1

File 2: test2.txt

This is test file 2

Save both files.

Your project structure:

DataScienceProject5/

|

├── program1.py

├── program2

|

└── IGNORE/

  ├── test1.txt

  └── test2.txt

Step 11. Create .gitignore file

Return to the main project folder.

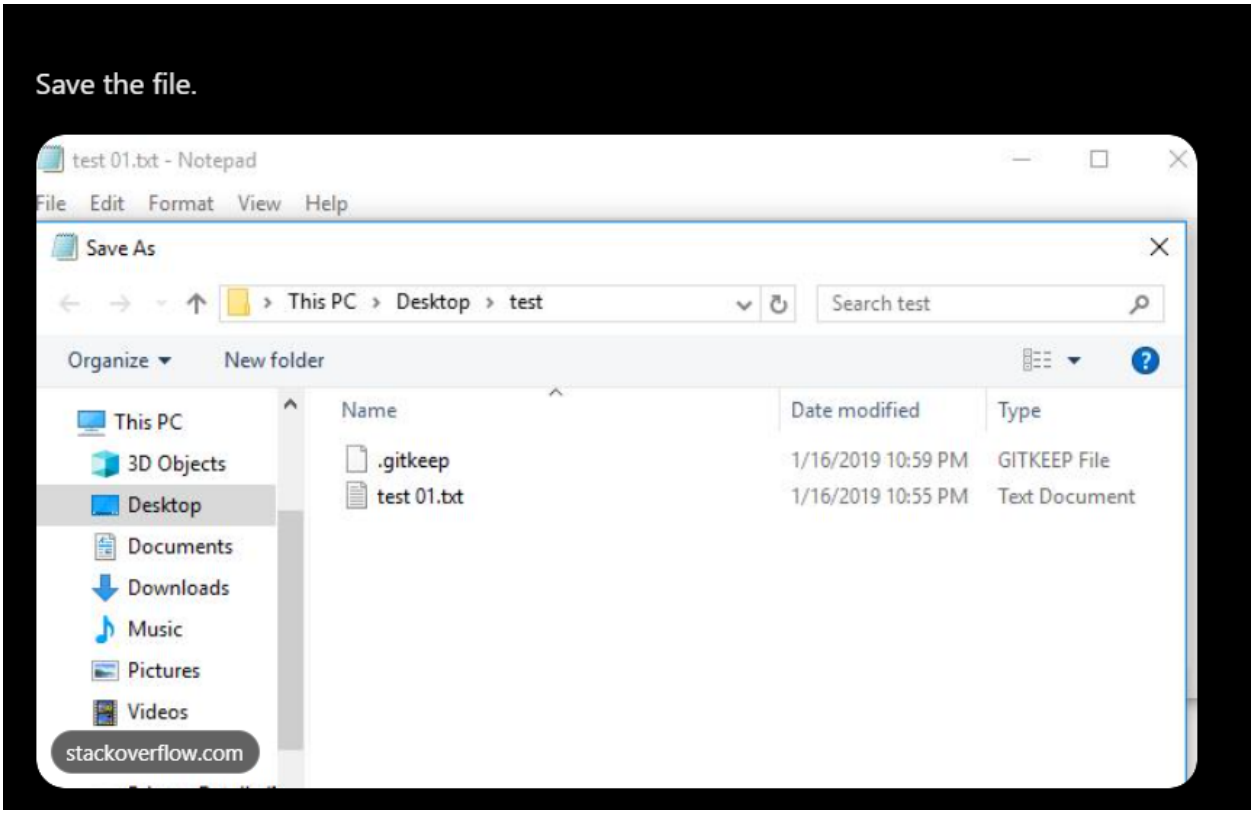
Create a new text file named:

.gitignore

Make sure it is not accidentally saved as .gitignore.txt.

Open it and write:

IGNORE/



Save the file.

[git - How to create a .gitignore file - Stack Overflow]

[stackoverflow.com]

Meaning: Git will ignore the complete IGNORE folder and its contents.

Step 12. Check changes in GitHub Desktop

Return to GitHub Desktop.

You should see .gitignore in the Changes tab.

The IGNORE folder should not appear as an ordinary changed file.

Why?

Because .gitignore tells Git not to track the IGNORE folder.

Step 13. Commit and push .gitignore

In GitHub Desktop:

1. Select .gitignore.
2. Enter commit summary:

Add gitignore to ignore IGNORE directory

3. Click Commit to GIT-EXAM-1.
4. Click Push origin.

Your changes are now uploaded.

Step 14. Verify on GitHub

Open your GitHub repository.

Select branch GIT-EXAM-1.

Expected contents:

program1.py

Uploaded

program2

Uploaded

.gitignore

Uploaded

IGNORE/

Not uploaded

The IGNORE folder, test1.txt, and test2.txt should not appear on GitHub.

Experiment 5 completed

Important viva questions

Question	**Answer**

What is GitHub Desktop?	A graphical application for managing Git repositories.
What is Git?	A version control system.
What is GitHub?	An online platform for hosting Git repositories.
What is a repository?	A project folder tracked by Git.
What is a branch?	A separate line of development.
What is commit?	Saving changes in Git history.
What is push?	Uploading local commits to GitHub.
What is .gitignore?	A file used to ignore specified files and folders.
Why use .gitignore?	To prevent unwanted files from being tracked.
What is Publish repository?	Uploading a new local repository to GitHub.

Experiment 1–5: What you should remember

Experiment	Main concept
1	Git basics, multiplication table, greatest of two numbers
2	Even/odd, reverse string, Pull Request
3	Two branches, pulling and merge conflicts
4	.gitignore using Git Bash
5	GitHub Desktop and .gitignore

Next: Ubuntu practicals

Experiments 6 onwards

Now we will switch from Windows Git Bash to Ubuntu.

Before starting Experiment 6, I need to know one thing:

Do you already have Ubuntu installed on your computer?

Yes, Ubuntu is installed in VirtualBox / VMware.

Yes, I have Ubuntu alongside Windows (dual boot).

I use Ubuntu through WSL on Windows.

No, I don't have Ubuntu installed.

I don't know.

Continue to Ubuntu experiments

We'll keep the same exam-focused approach: exact steps, commands, expected output, explanations and viva questions.

Experiment 6: Forking, Branching and Pull Request using GitHub Desktop

Windows – GitHub Desktop + Git Bash

Exam practical

In this experiment, we work with another person's GitHub repository, make our own copy, create a new branch, modify code and submit a Pull Request.

Repository given in your assignment:

AI-ProjectGallery – GitHub repository

1. Understand the basic concepts

Term	**Simple meaning**
Clone	Download a repository to your computer.
Fork	Create your own copy of someone else's repository on GitHub.
Branch	Create a separate workspace for changes.
Feature branch	A branch created to implement a new feature.
Pull Request (PR)	Request to contribute your changes to the original project.
Merge conflict	When Git cannot automatically combine different changes.

Workflow of this experiment:

Original repository

KalyanM45 / AI-ProjectGallery

Fork

Your GitHub repository

YourUsername / AI-ProjectGallery

Clone and create branch

GIT-EXAM

Make code changes and commit

Pull Request

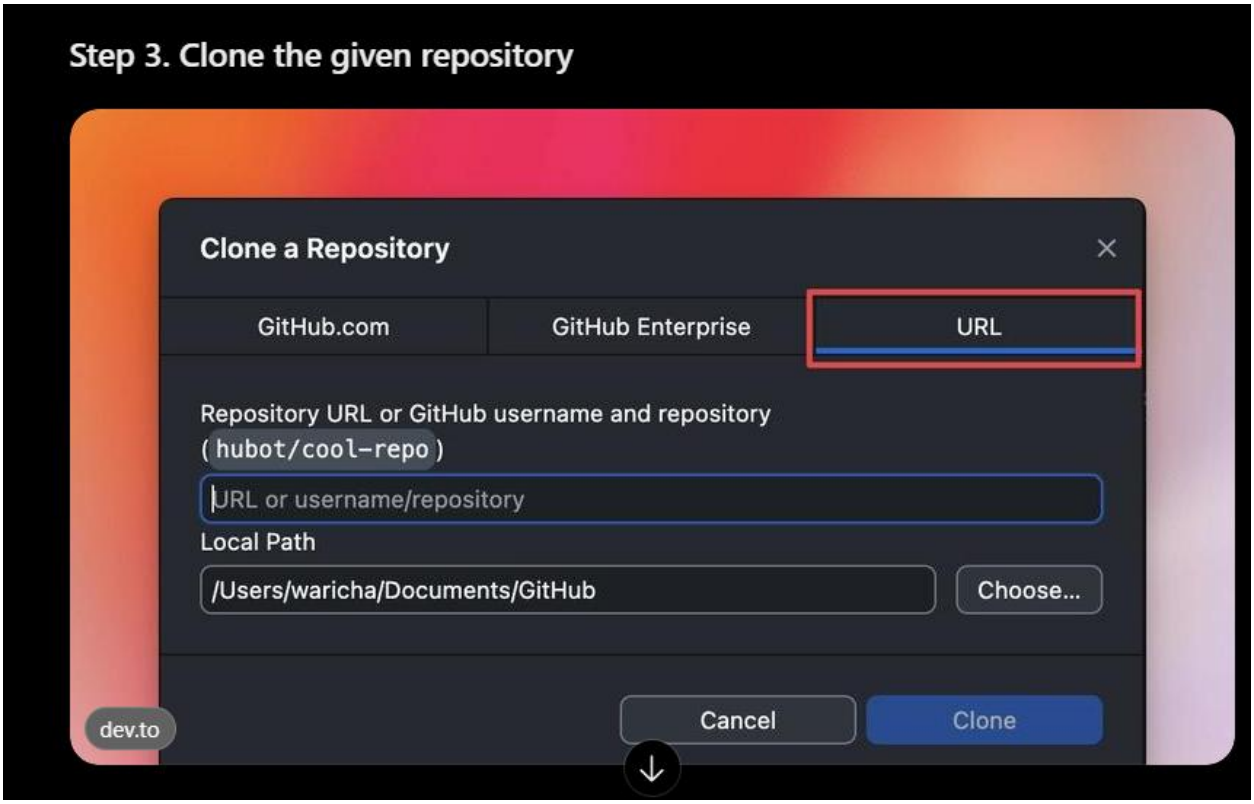
Original repository

Review and possible merge

Step 2. Open GitHub Desktop

1. Open GitHub Desktop on Windows.
2. Sign in to your GitHub account.
3. Make sure your account is connected.

Step 3. Clone the given repository



[A Step-by-Step Guide to Cloning a GitHub Repository to Your Local Machine - DEV Community

dev.to]

1. Open GitHub Desktop.
2. Click File → Clone Repository.
3. Select the URL tab.
4. Paste this URL:

https://github.com/KalyanM45/AI-ProjectGallery.git

5. Choose a local path, for example Desktop.
6. Click Clone.

The repository will be downloaded to your computer.

Step 4. Demonstrate that branch creation is disallowed

The original repository belongs to another user.

1. Open the cloned repository in GitHub Desktop.
2. Select Current Branch.
3. Try selecting New Branch.
4. If GitHub does not allow you to publish a branch to the original repository, this demonstrates the permission restriction.

Important: Git may allow you to create a local branch, even when you do not have permission to push it to the original repository. The restriction is on publishing changes to the owner's remote repository, not necessarily creating a local branch.

Important: Correct repository link

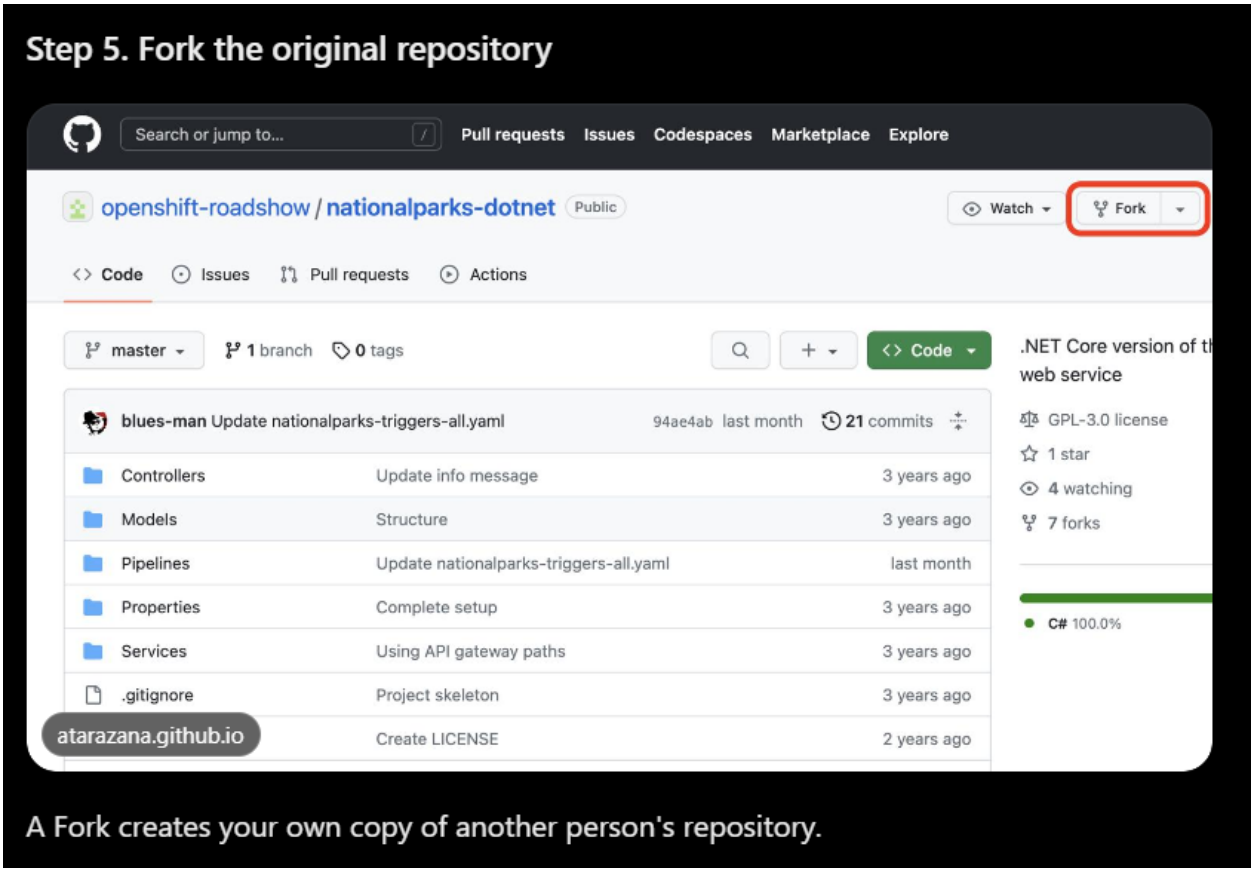
I checked the repository mentioned in your assignment. Its actual GitHub name is AI-Project-Gallery (with a hyphen before Gallery), and its default branch is main. The repository contains a README.md file listing AI projects.

GitHub+1

Use this corrected link:

Open original AI-Project-Gallery repository

Step 5. Fork the original repository



[Webhooks with OpenShift :: OpenShift Starter Guides]

[atarazana.github.io]

A Fork creates your own copy of another person's repository.

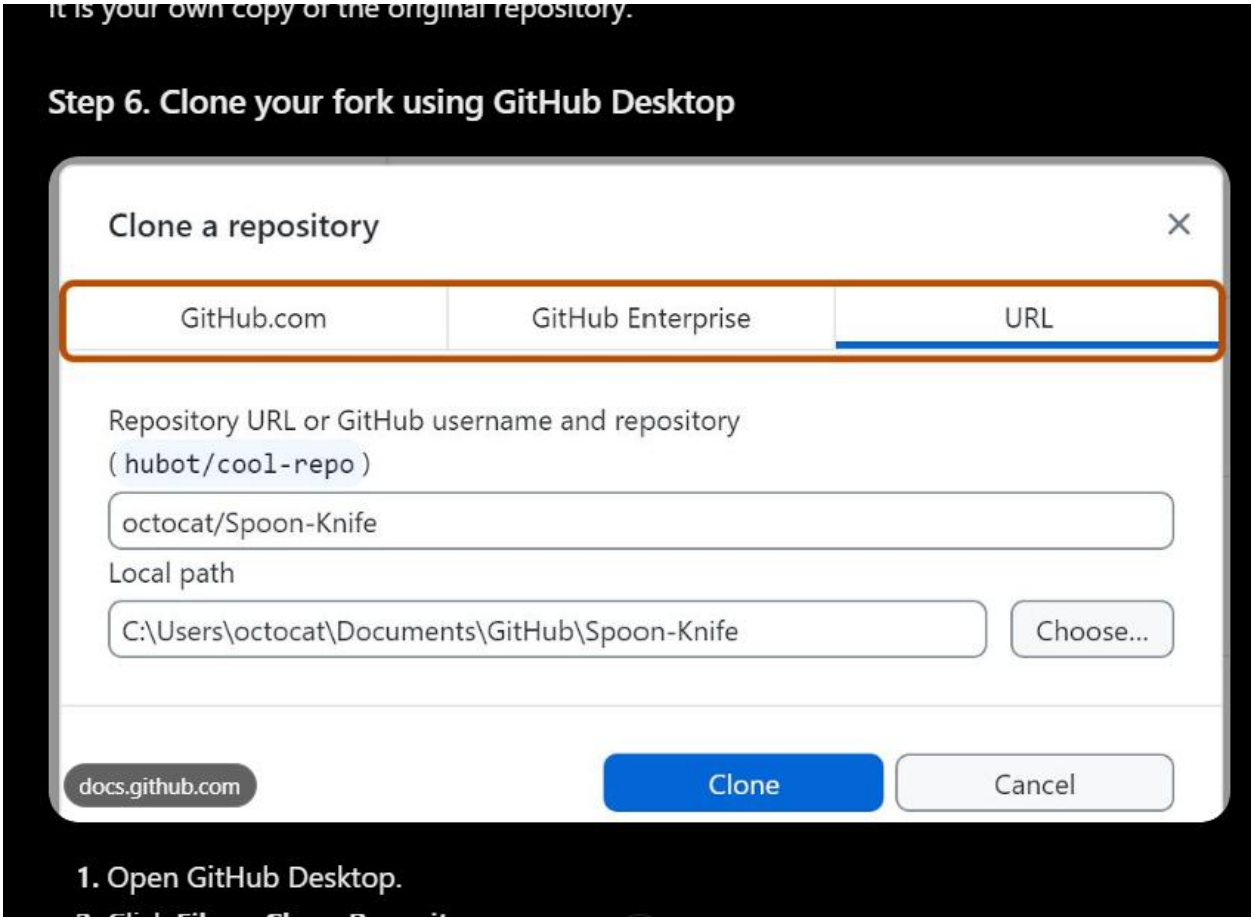
1. Open the original repository on GitHub.
2. Click Fork (top-right corner).
3. Repository name: AI-Project-Gallery.
4. Owner: select your GitHub username.
5. Click Create fork.

Now your repository will look like:

YOUR-USERNAME/AI-Project-Gallery

It is your own copy of the original repository.

Step 6. Clone your fork using GitHub Desktop



[Cloning and forking repositories from GitHub Desktop - GitHub Docs](docs.github.com)

docs.github.com

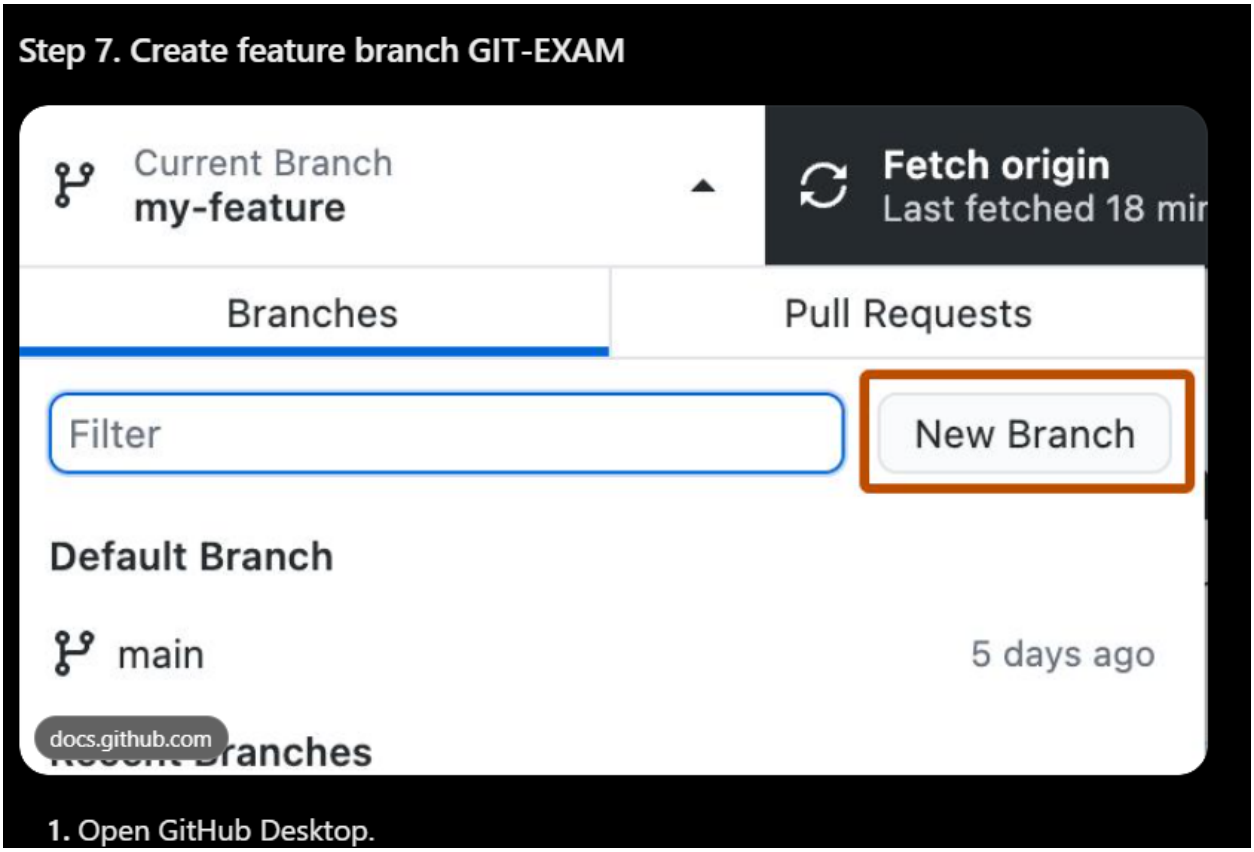
1. Open GitHub Desktop.
2. Click File → Clone Repository.
3. Select URL.
4. Paste your fork's URL:

https://github.com/YOUR-USERNAME/AI-Project-Gallery.git

5. Choose a local folder.
6. Click Clone.

Important: Clone your fork, not the original repository, for the remaining practical work.

Step 7. Create feature branch GIT-EXAM



[Managing branches in GitHub Desktop - GitHub Docs]

[docs.github.com]

1. Open GitHub Desktop.
2. Click Current Branch.
3. Click New Branch.
4. Enter:

GIT-EXAM

5. Click Create Branch.

Now your current branch is GIT-EXAM.

Step 8. Make changes in the branch

The repository mainly contains README.md, so we will modify that file.

1. In GitHub Desktop, click Repository → Show in Explorer.
2. Open README.md using Notepad.

3. Go to the bottom of the file.
4. Add this content:

GIT-EXAM Contribution

This section was added using the GIT-EXAM feature branch.

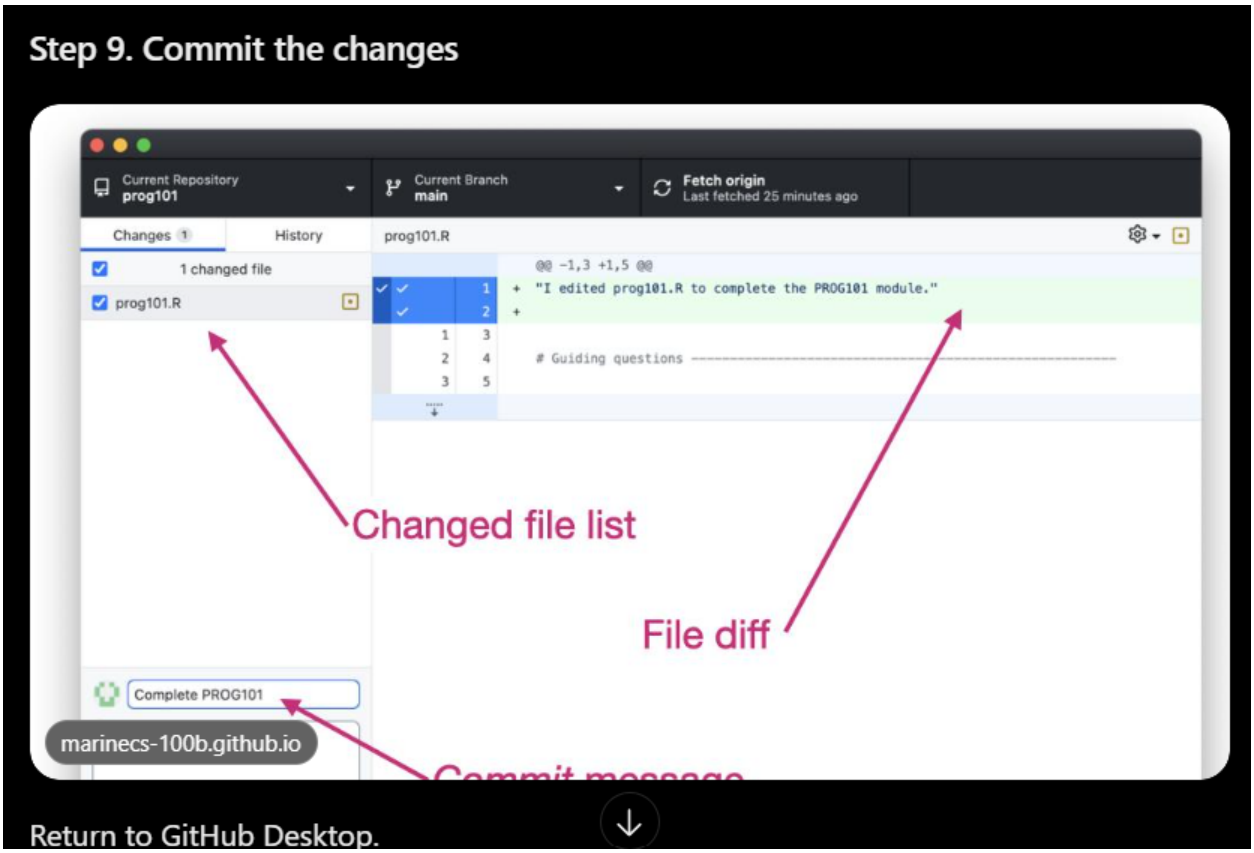
New Project Idea

- Project Name: Student Performance Prediction
- Domain: Machine Learning
- Description: Predict student performance using academic data.

5. Save and close the file.

This is your code/documentation change for the practical.

Step 9. Commit the changes



[Intro to Marine Data Science – Module Setup and Submission]

[marinecs-100b.github.io]

Return to GitHub Desktop.

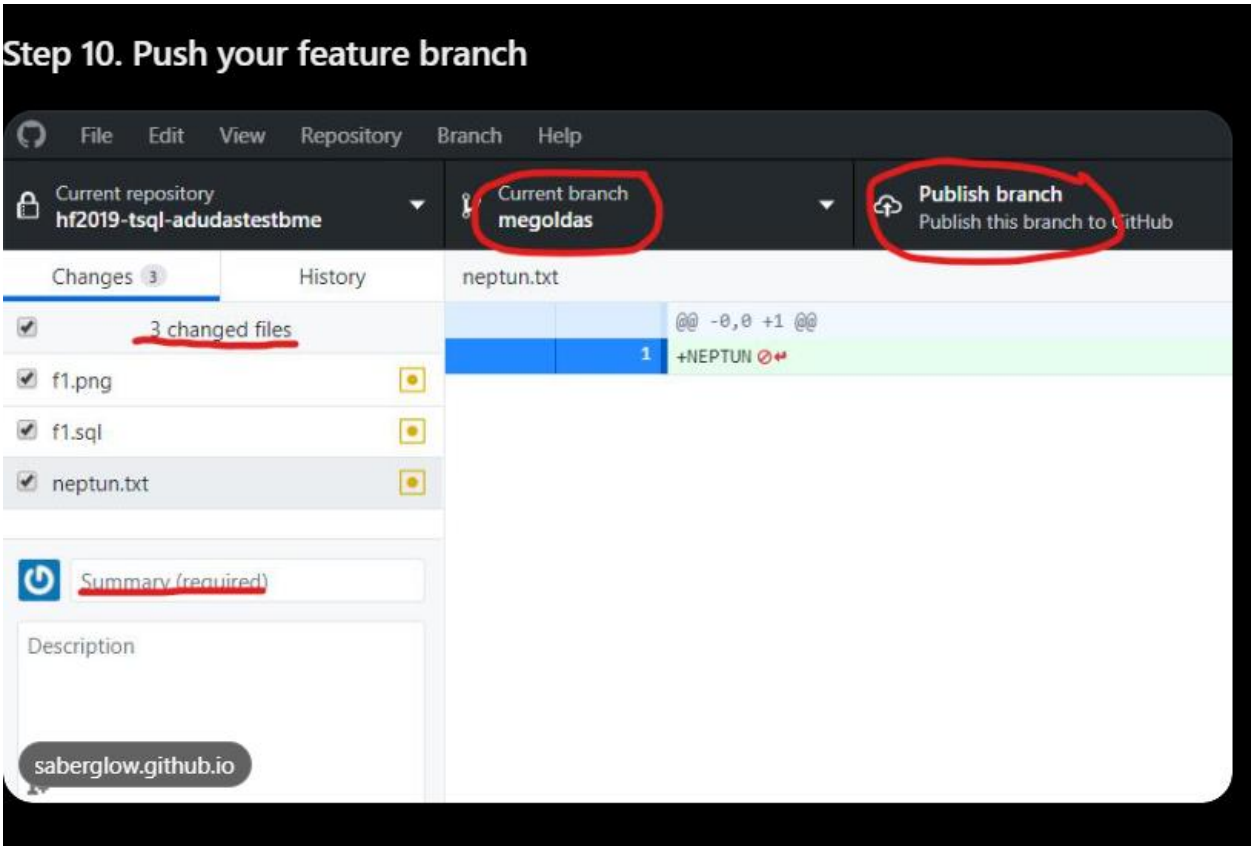
1. Open the Changes tab.
2. You should see README.md.
3. In Summary, write:

Add student performance prediction project

4. Click Commit to GIT-EXAM.

A commit saves your changes in Git.

Step 10. Push your feature branch



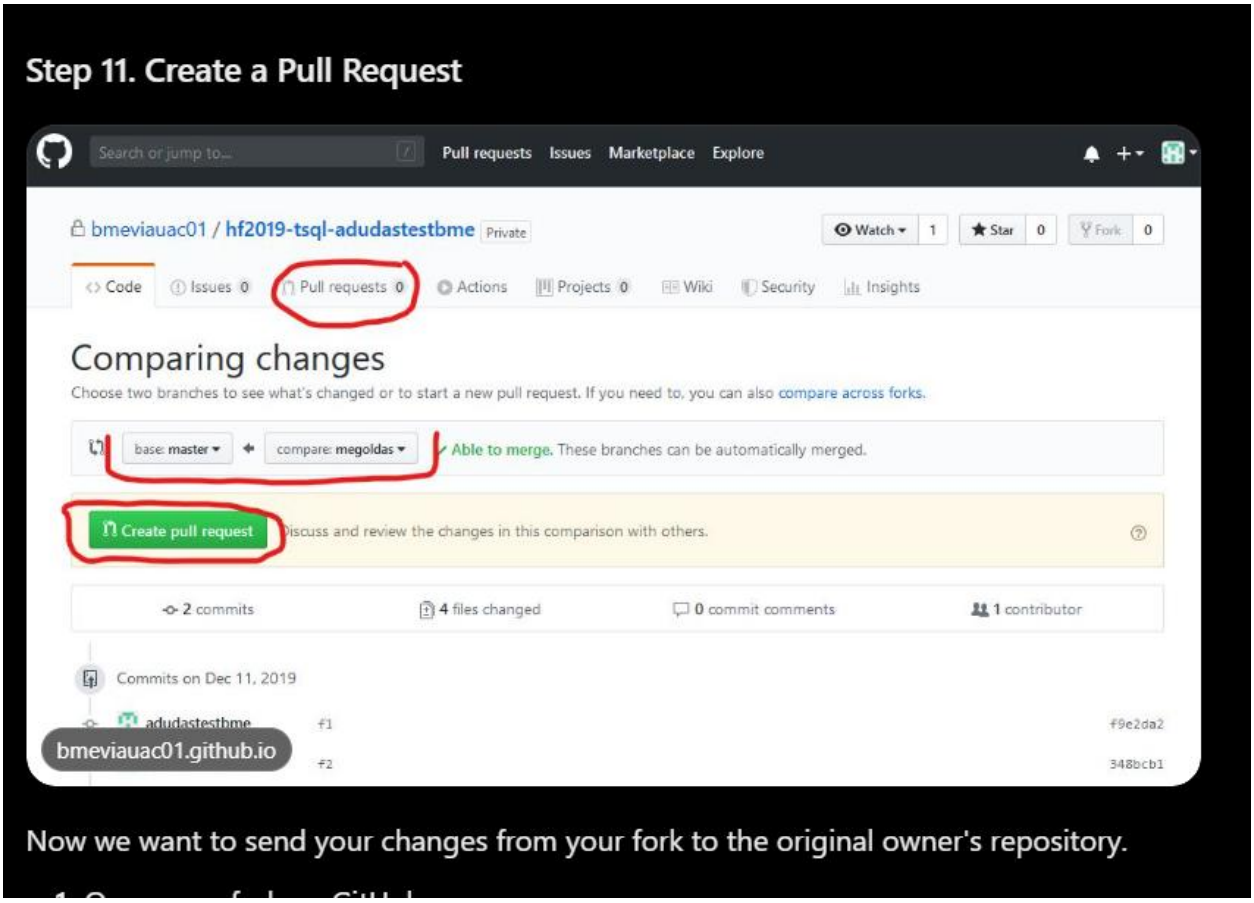
[Submitting your work (GitHub) - BMEVIAUAC09 Software development laboratory 1

saberglow.github.io](saberglow.github.io)

1. Click Publish branch or Push origin.
2. Wait for the upload.
3. Open your fork on GitHub.
4. Select branch GIT-EXAM.
5. Open README.md.

You should see your newly added contribution.

Step 11. Create a Pull Request



[Submitting your work (GitHub) - BMEVIAUAC16 Data-Driven Software Development Lab](bmeviauac01.github.io)

Now we want to send your changes from your fork to the original owner's repository.

1. Open your fork on GitHub.
2. Click Contribute or Compare & pull request.
3. Select the following settings:

Base repository (original)

KalyanM45/AI-Project-Gallery

Base branch

main

Head repository (your fork)

YOUR-USERNAME/AI-Project-Gallery

Compare branch

GIT-EXAM

Your GIT-EXAM → Original main

4. Enter Pull Request title:

Add Student Performance Prediction Project

5. Description:

This pull request adds a new Student Performance

Prediction project entry through the GIT-EXAM branch.

6. Click Create pull request.

Your Pull Request has been submitted for review.

Step 12. Resolve review feedback

After creating a Pull Request, the repository owner or reviewer may review your changes.

If the reviewer requests changes:

1. Open your Pull Request.
2. Read the review comments.
3. Open your local project folder.
4. Modify README.md according to the feedback.
5. Save the file.
6. Return to GitHub Desktop.
7. Enter commit summary:

Update README based on review feedback

8. Click Commit to GIT-EXAM.
9. Click Push origin.

Your existing Pull Request automatically gets updated with the new commit.

Step 13. Resolve merge conflicts

Only if a conflict occurs

Sometimes GitHub displays:

This branch has conflicts that must be resolved

To resolve conflicts, open Git Bash inside your cloned fork folder.

First, connect the original repository as an upstream remote:

git remote add upstream https://github.com/KalyanM45/AI-Project-Gallery.git

Download its latest changes:

git fetch upstream

Switch to your feature branch:

git checkout GIT-EXAM

Merge original changes:

git merge upstream/main

If a conflict occurs, open the conflicted file:

notepad README.md

You may see:

<<<<<<< HEAD

Your changes

=======

Incoming changes

>>>>>>> upstream/main

Remove the conflict markers and keep the correct combined content.

Save the file.

Now execute:

git add README.md

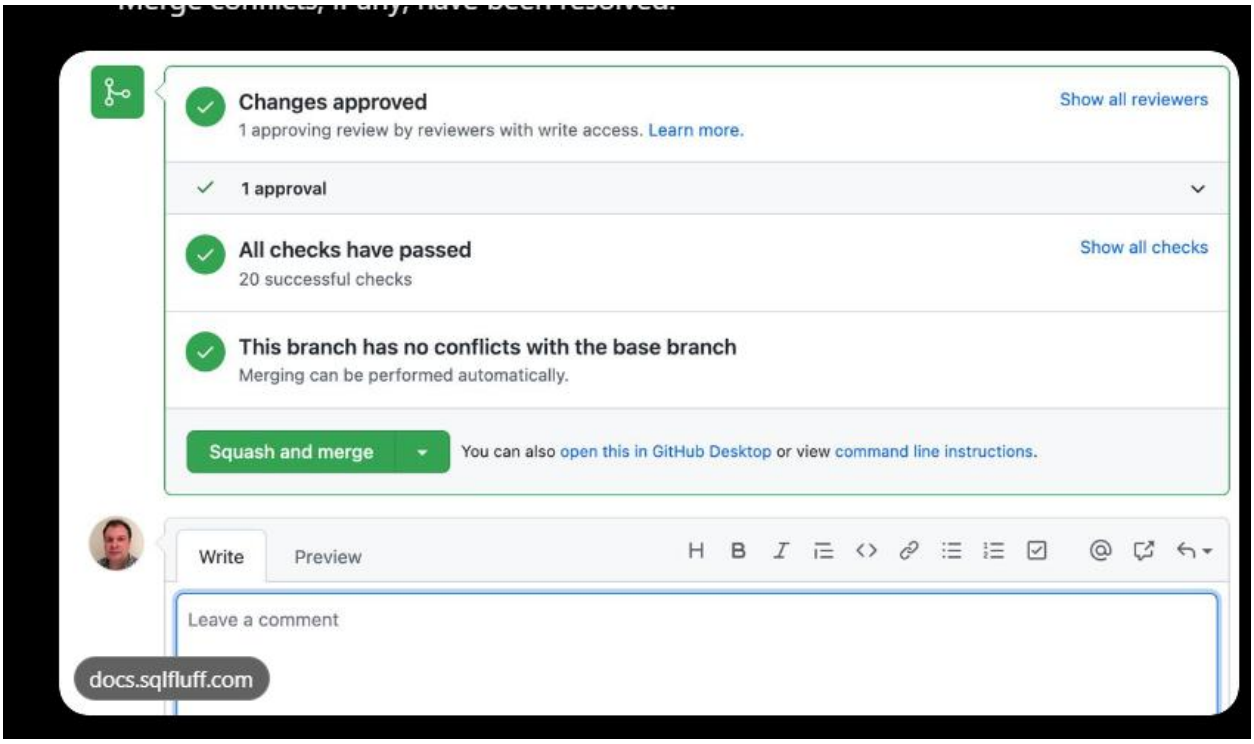
git commit -m "Resolve merge conflict"

git push origin GIT-EXAM

Your Pull Request will automatically update.

Remember: A conflict is not guaranteed to happen in every Pull Request. If GitHub reports no conflicts, mention that the changes can be merged automatically.

Step 14. Final verification



Open your Pull Request on GitHub.

Verify:

- Your feature branch is GIT-EXAM.
- Your changes are visible.
- Your commits are visible.
- Review comments, if any, have been addressed.
- Merge conflicts, if any, have been resolved.

[How to use Git — SQLFluff stable version documentation]

[docs.sqlfluff.com]

Experiment 6 practical steps completed

Important: You can submit the Pull Request, but the original repository owner must approve and merge it. You cannot force a merge into someone else's repository.

Important viva questions

Question	Answer
What is cloning?	Downloading a remote repository to a local computer.
What is forking?	Creating your own copy of another user's repository.
Why do we fork?	To contribute without direct write permission.
What is a feature branch?	A separate branch for implementing new features.
What is a Pull Request?	A request to merge changes into another repository or branch.
What is upstream?	The original repository from which a fork was created.
What is origin?	The remote repository associated with your fork.
What is merge conflict?	A situation where Git cannot automatically combine changes.
How do you resolve conflicts?	Edit the conflicting file, stage it, commit and push.
Who approves a Pull Request?	The repository owner or authorized reviewer.

Experiment 6 command revision

These are the important Git Bash commands in case your examiner asks you to demonstrate them:

git clone https://github.com/KalyanM45/AI-Project-Gallery.git

git checkout -b GIT-EXAM

git remote add upstream https://github.com/KalyanM45/AI-Project-Gallery.git

git fetch upstream

git merge upstream/main

git add README.md

git commit -m "Add new project"

git push origin GIT-EXAM

Pull Request creation is done through GitHub.

Your Git practical preparation is now complete!

Windows experiments

1. Git basics – Multiplication and greatest number

2. Even/Odd, reverse string and Pull Request

3. Branching and merge conflicts

4. .gitignore using Git Bash

5. GitHub Desktop and .gitignore

6. Fork, feature branch and Pull Request

Next: Ubuntu experiments

We will start from opening Ubuntu Terminal, basic Linux commands, creating files and folders, and then perform each assignment.

One last thing: Before we move to Experiment 7, tell me whether your Ubuntu is installed in VirtualBox, VMware, WSL, or not installed yet.

Experiment 7: Clone, Fork, Branch and Pull Request using GitHub Desktop

Windows practical

Software required: GitHub Desktop, Git Bash (for conflict resolution), and a GitHub account.

Your assignment gives this repository:

https://github.com/KalyanM45/Spam-EmailDetection.git

Small correction: The repository appears on GitHub under the name Spam-Email-Detection (with a hyphen between Spam, Email and Detection). Use this link:

Open Spam-Email-Detection repository

GitHub

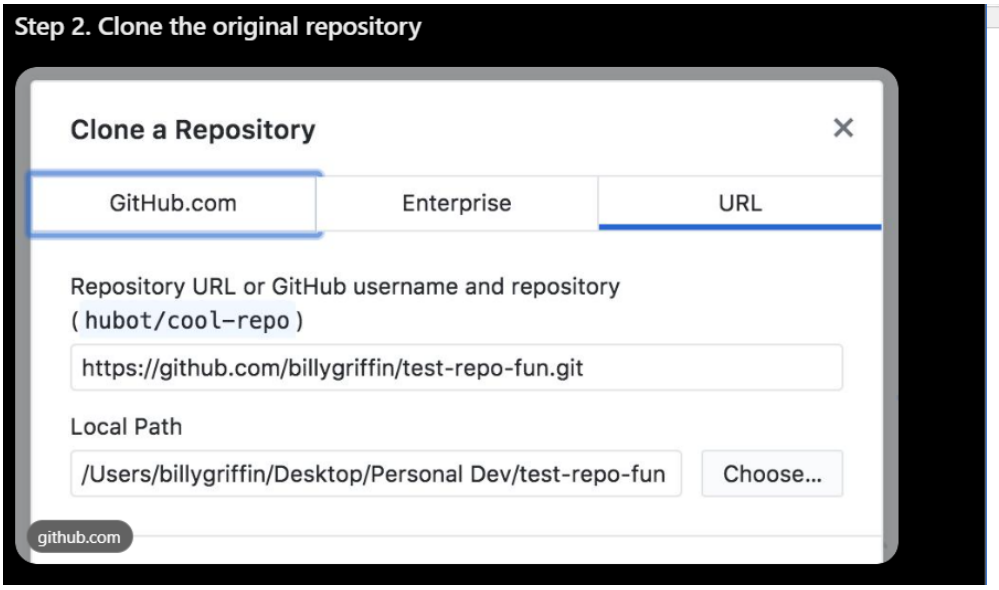
The repository is a Python machine-learning spam/ham email classifier with a Streamlit interface.

GitHub

Step 1. Open GitHub Desktop

1. Open GitHub Desktop on Windows.
2. Sign in to your GitHub account.
3. Keep the application open.

Step 2. Clone the original repository



[Open in Desktop launches GitHub Desktop but doesn't start clone process · Issue #5324 · desktop/desktop · GitHub

github.com](github.com)

1. Click File → Clone Repository.
2. Select the URL tab.
3. Paste:

https://github.com/KalyanM45/Spam-Email-Detection.git

4. Choose a local path, such as your Desktop.
5. Click Clone.

The repository is now downloaded to your computer.

Step 3. Demonstrate that branch publishing is disallowed

The original repository belongs to another GitHub user.

1. In GitHub Desktop, click Current Branch → New Branch.
2. Enter:

GIT-EXAM

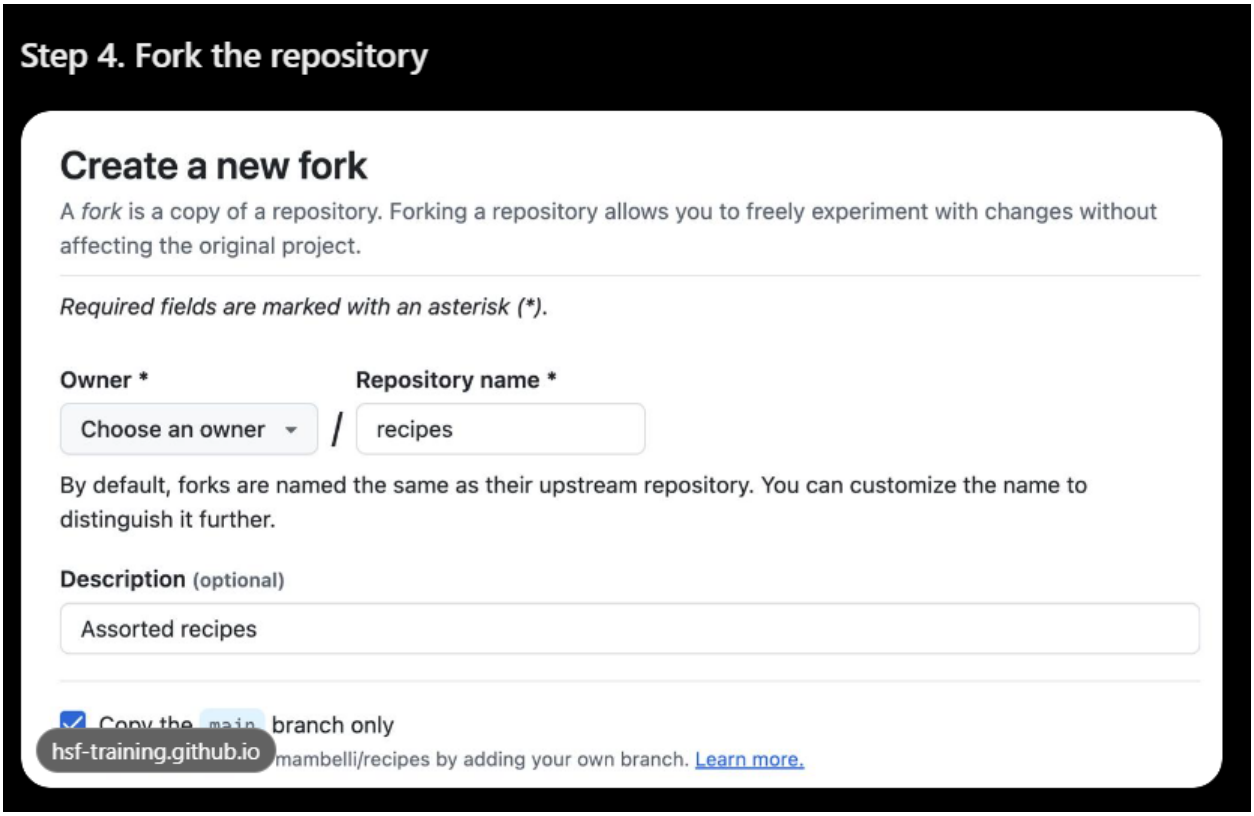
3. Click Create Branch.

This creates a local branch. Now try to publish/push it to the original repository if the option is available.

If you do not have write permission, GitHub will reject the push with a permission error.

Viva point: Creating a local branch is allowed; pushing a branch to someone else's repository requires write permission. That is why we fork the repository.

Step 4. Fork the repository



[Version Control with Git: Collaborating - Branching and Pull Requests]

[hsf-training.github.io]

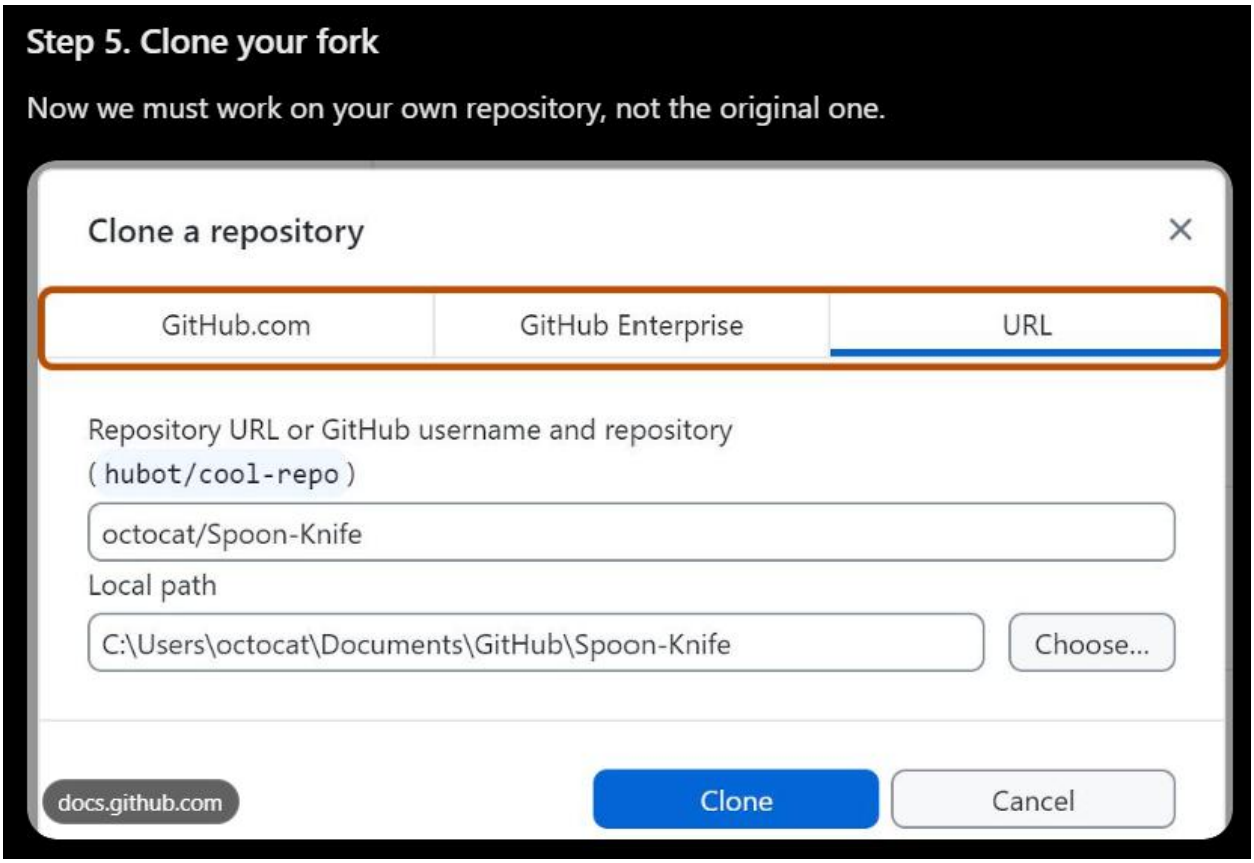
1. Open the original repository on GitHub.
2. Click Fork.
3. Select your GitHub account as the owner.
4. Keep the repository name Spam-Email-Detection.
5. Click Create fork.

Your fork will have this address:

https://github.com/YOUR-USERNAME/Spam-Email-Detection

Replace YOUR-USERNAME with your actual GitHub username.

Step 5. Clone your fork



Now we must work on your own repository, not the original one.

[Cloning and forking repositories from GitHub Desktop - GitHub Docs]

[docs.github.com]

1. Open GitHub Desktop.
2. Click File → Clone Repository.
3. Select URL.
4. Paste your fork URL:

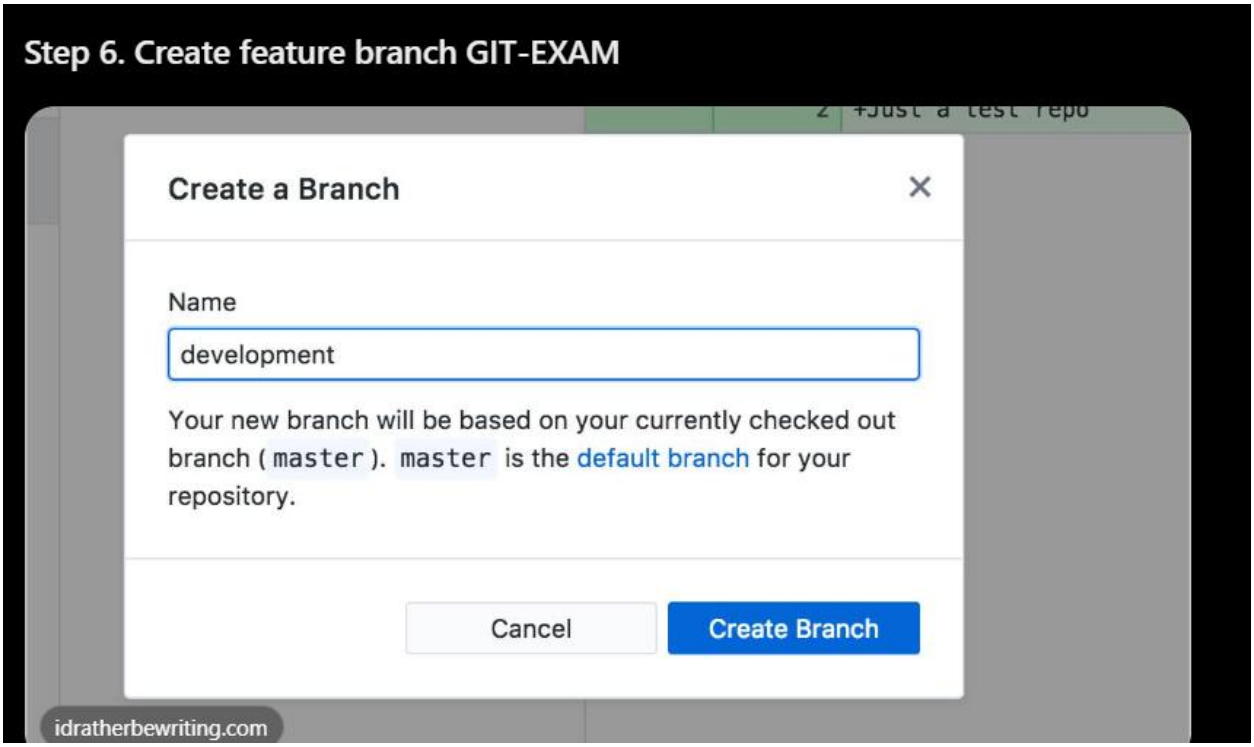
https://github.com/YOUR-USERNAME/Spam-Email-Detection.git

5. Choose a different local folder, such as:

Desktop/Spam-Email-Detection-Fork

6. Click Clone.

Step 6. Create feature branch GIT-EXAM



[Activity: Use the GitHub Desktop client | I'd Rather Be Writing Blog and API doc course](idratherbewriting.com)

1. Click Current Branch.
2. Click New Branch.
3. Enter:

GIT-EXAM

4. Click Create Branch.

Your current branch should now show:

GIT-EXAM

Step 7. Add your code

We will add a new Python file for a simple spam email demonstration. This will not overwrite the existing machine-learning project files.

1. In GitHub Desktop, click Repository → Show in Explorer.
2. In the project folder, create a new file named:

git_exam_spam_demo.py

3. Open it in Notepad or VS Code.
4. Copy this code:

Simple Spam Email Detection Demoemail = input("Enter email message: ")spam_words = ["win", "free", "prize", "offer", "urgent", "claim"]email = email.lower()spam_count = 0for word in spam_words: if word in email: spam_count += 1if spam_count >= 2: print("Result: Spam Email")else: print("Result: Ham (Not Spam) Email")

5. Save the file.

Example output:

Enter email message: Win a free prize now

Result: Spam Email

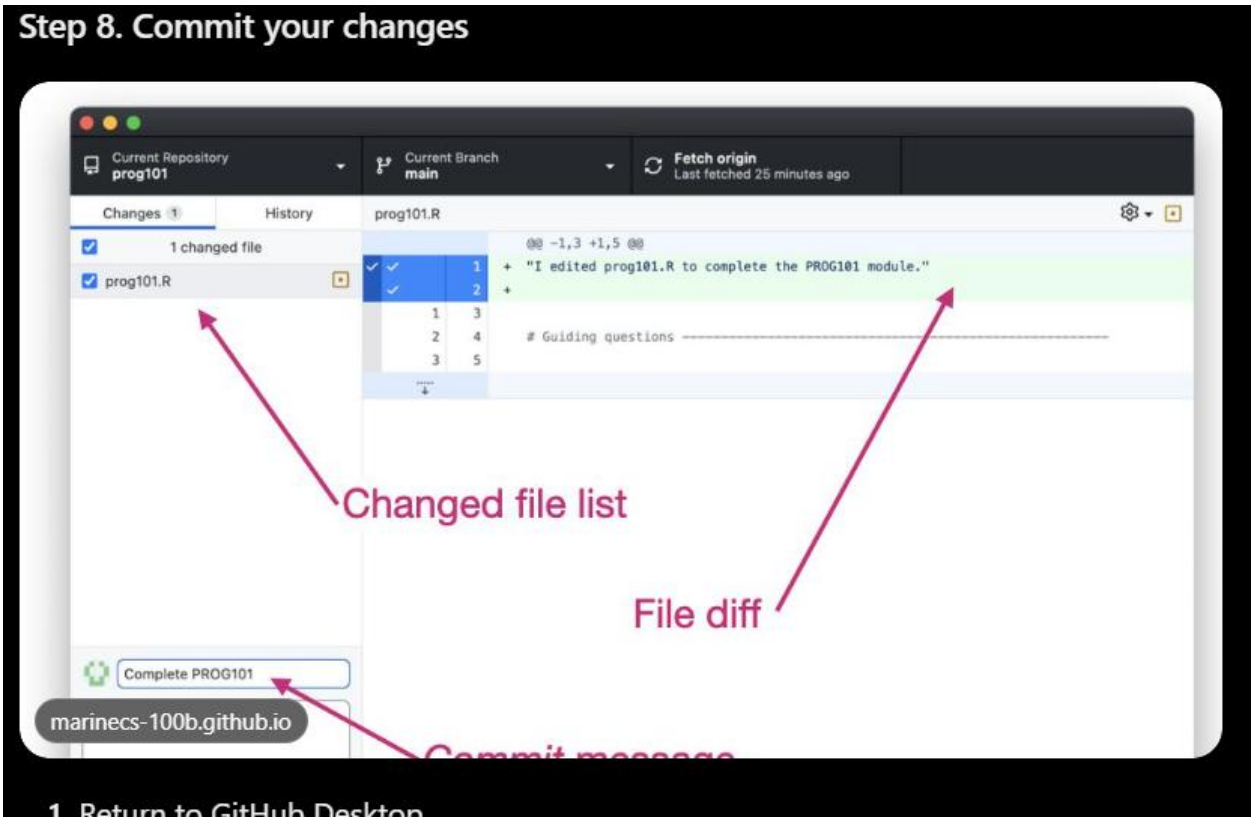
Another example:

Enter email message: Hello, please send the meeting notes

Result: Ham (Not Spam) Email

This is a simple keyword-based demonstration for your Git contribution, not the repository's trained ML model.

Step 8. Commit your changes



[Intro to Marine Data Science – Module Setup and Submission]

[marinecs-100b.github.io]

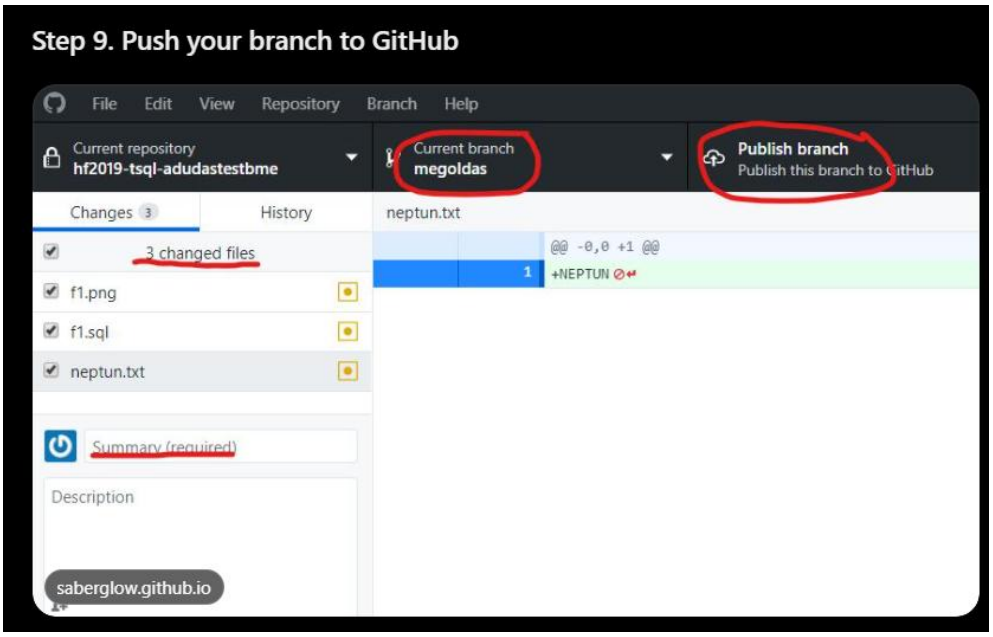
1. Return to GitHub Desktop.
2. Click Changes.
3. You should see git_exam_spam_demo.py.
4. In the Summary field, type:

Add spam email detection demo

5. Click Commit to GIT-EXAM.

Your change is now committed to your feature branch.

Step 9. Push your branch to GitHub



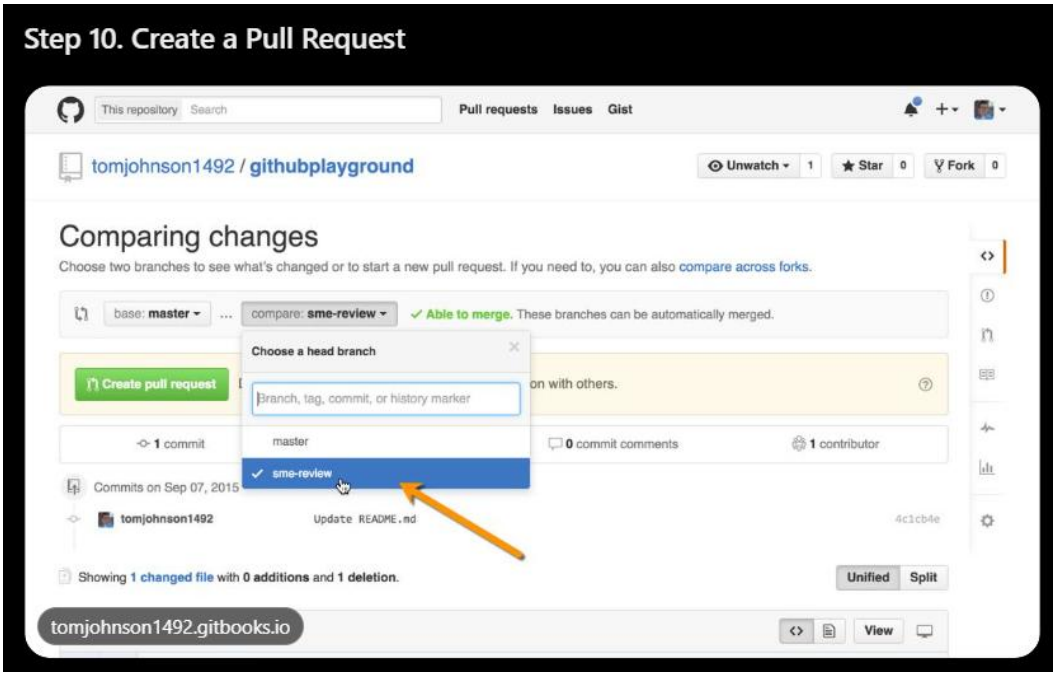
[Submitting your work (GitHub) - BMEVIAUAC09 Software development laboratory 1

saberglow.github.io]

1. Click Publish branch or Push origin.
2. Wait until the upload finishes.
3. Open your fork on GitHub.
4. Select branch GIT-EXAM.
5. Check that git_exam_spam_demo.py is visible.

Your feature branch and code are now uploaded.

Step 10. Create a Pull Request



[Pull request workflows through Github in browser · Documenting REST APIs](tomjohnson1492.gitbooks.io)

tomjohnson1492.gitbooks.io

1. Open your fork on GitHub.
2. Click Contribute → Open pull request or Compare & pull request.
3. Set the following:

Setting	**Select**
Base repository	KalyanM45/Spam-Email-Detection
Base branch	main
Head repository	YOUR-USERNAME/Spam-Email-Detection
Compare branch	GIT-EXAM

4. Enter the title:

Add spam email detection demo

5. Description:

This pull request adds a simple Python

spam email detection demonstration.

It uses keyword matching to classify

messages as Spam or Ham.

6. Click Create pull request.

Pull Request submitted

Your changes have been sent to the original repository owner for review. The owner decides whether to approve and merge them.

Step 11. Incorporate review feedback

If the reviewer asks you to make changes:

1. Open your Pull Request on GitHub.
2. Read the reviewer's comments.
3. Open your local fork project folder.
4. Edit git_exam_spam_demo.py as requested.
5. Save the file.
6. Open GitHub Desktop.
7. Enter commit summary:

Update spam detection code based on review

8. Click Commit to GIT-EXAM.
9. Click Push origin.

Your existing Pull Request will automatically show the new changes.

Step 12. Resolve merge conflicts

If a conflict occurs

A merge conflict occurs when Git cannot automatically combine two changes.

First, open Git Bash inside your fork's project folder.

1. Connect the original repository:

git remote add upstream https://github.com/KalyanM45/Spam-Email-Detection.git

2. Download the latest original repository changes:

git fetch upstream

3. Switch to your feature branch:

git checkout GIT-EXAM

4. Merge original repository changes:

git merge upstream/main

If a conflict occurs, Git may display:

CONFLICT (content): Merge conflict in README.md

5. Open the conflicted file:

notepad README.md

You may see:

<<<<<<< HEAD

Your changes

=======

Original repository changes

>>>>>>> upstream/main

6. Resolve the conflict:

Remove these three conflict markers:

- <<<<<<< HEAD
- =======
- >>>>>>> upstream/main

Keep the correct content from both versions where appropriate, then save the file.

7. Complete the merge:

git add README.md

git commit -m "Resolve merge conflict"

git push origin GIT-EXAM

Your Pull Request will automatically update.

If no conflict occurs, Git will complete the merge automatically. Do not create a fake conflict just to show this step.

Step 13. Final verification

Open your fork and Pull Request on GitHub. Verify:

- Original repository cloned.
- Fork created successfully.
- Feature branch GIT-EXAM created.
- Python file added.
- Changes committed.
- Branch pushed to GitHub.
- Pull Request submitted.
- Review feedback incorporated, if received.
- Merge conflicts resolved, if they occurred.

Important viva questions

Question	Answer
What is GitHub Desktop?	A graphical application for managing Git repositories.
What is cloning?	Downloading a remote repository to a computer.
What is forking?	Creating your own copy of another user's repository.
Why do we fork?	To contribute without write access to the original repository.
What is a feature branch?	A separate branch used to develop a feature.
What is a Pull Request?	A request to merge changes into another branch or repository.
What is spam email?	An unwanted or unsolicited email.
What is ham email?	A legitimate, non-spam email.
What is a merge conflict?	A problem when Git cannot automatically combine changes.

What is upstream?	The original repository from which a fork was created.
What is origin?	The remote repository of your fork.

Important commands for revision

git clone https://github.com/KalyanM45/Spam-Email-Detection.git

git checkout -b GIT-EXAM

git remote add upstream https://github.com/KalyanM45/Spam-Email-Detection.git

git fetch upstream

git merge upstream/main

git add .

git commit -m "Add spam detection demo"

git push origin GIT-EXAM

Remember: Git Bash commands are optional for most of this practical. You can do cloning, branching, committing and pushing through GitHub Desktop. Pull Request creation and review happen on GitHub.